边缘 AI 全链路实战

从零训练到量化部署的 11 课

副标题 从 PyTorch 训练到 INT8 量化部署的完整实践

包含 11 课讲义 · 全部实测结果 · 可复现代码 · 39 道面试题

硬件 RTX 5060 Ti 8GB (sm_120) · 12 核 CPU

生成日期 2026 年 10 月 17 日

92.18%	2.03e-06	14.89x	±3 pp
CIFAR-10 测试准确率	ONNX 导出对齐误差	模型最大压缩倍数	实验噪声底线

全部数字来自本仓库的实测结果，可在各课 out/ 目录下复现。

目录

总览	**课程总览**	路线图与进度	3
第 1 课	**训练循环的五件事**	张量 / DataLoader / autograd / optimizer	10
第 2 课	**从 MLP 到 CNN**	把准确率从 53% 做到 92%	15
第 3 课	**导出 ONNX**	训练框架与推理框架的分水岭	22
第 4 课	**延迟与内存基准测试**	把「跑多快」变成可信的数字	29
第 5 课	**部署到真实目标硬件**	等待硬件，暂未完成	37
第 6 课	**PTQ 训练后量化**	把 FP32 压成 INT8	38
第 7 课	**QAT 量化感知训练**	让模型提前适应低精度	45
第 8 课	**剪枝与知识蒸馏**	压缩的第三根支柱	52
第 9 课	**做你自己的数据集**	数据泄漏与域偏移	60
第 10 课	**迁移学习与微调**	只有几百张图怎么办	66
第 11 课	**端到端 pipeline 与作品集**	一条命令跑完全流程	73
附录	**附录：作品集与面试题库**	简历描述 + 39 道面试题	79

总览

课程总览

目标：学完具备投递边缘 AI 岗位的能力——能独立完成
数据 → 微调 → 量化 → 部署 → 报出延迟/精度 全链路。

怎么用这份路线

- **每课都有"动手交付物"**。看完不写代码 = 没学。交付物会积累成你的作品集。
- **一次只做一课**。每课的验收标准都写清楚了，达不到就不要往下走。
- **每课结束都有一句"简历上能写什么"**。11 课之后你自然有一份项目经历。
- 建议节奏：每课 5–10 小时（含动手），每周 1–2 课，**总共 10–14 周**。

课程表调整说明（2026-10-16）：原第 3 课"训练诊断与实验管理"已取消独立排课——
它的核心内容（读 loss 曲线、过拟合判断、对照实验定位问题）已经在**第 2 课的真实调试案例**
里完整讲透了，再单独开一课是重复。后面的课全部前移一位。

阶段一：PyTorch 与训练基础

✅ 第 1 课：训练循环的五件事

学什么	Tensor、Dataset/DataLoader、nn.Module、loss + autograd、optimizer
动手	用 MLP 在 CIFAR-10 上跑通完整训练循环
验收	能指着代码说出每一步在干什么（实测 **test acc 53.33%**，1.71M 参数 / FP32 6.51 MB）
已观察到	train acc 涨到 74% 而 test acc 卡在 53% —— 这就是**过拟合**
交付物	`lesson01/train_mlp.py`、`lesson01/README.md`（含 5 个作业）、标注版曲线图

✅ 第 2 课：从 MLP 到 CNN，把准确率做到 90%

学什么	卷积/池化为什么能学到空间特征；BatchNorm；数据增强；Dropout；学习率调度
动手	写一个 CNN，CIFAR-10 达到 ≥90%
验收	实测 **test acc 92.18%**，2.20M 参数，train/test 裂口从 21% 缩到 6%
简历上能写	"在 CIFAR-10 上训练 CNN 达到 **92.2%** 准确率"
⚠️ **本课踩坑**	学习率无预热 → loss 卡死在 **ln(10)=2.30**，准确率永远 10%。配套 `diagnose.py` 记录了标准排查流程（数据→初始输出→梯度→优化器）

阶段二：导出与部署 ← 边缘 AI 的分水岭

✅ 第 3 课：导出 ONNX + ONNX Runtime 推理

学什么	训练框架 vs 推理框架；ONNX 是什么；动态轴；**输出对齐验证**；warmup
动手	把第 2 课的模型导出成单文件 `.onnx`，用 onnxruntime 推理并逐项对齐
验收	最大输出误差 **2.03e-06**；2000 张测试图准确率 PyTorch/ONNX 都是 **0.9215**、预测一致率 **100%**
简历上能写	"将 PyTorch 模型导出为 ONNX 单文件（8.41 MB），完成数值对齐验证（误差 <1e-5）与 CPU 推理延迟基准（p50 0.404 ms）"
⚠️ **本课踩坑**	① 新导出器把权重写到外部 `.data` 文件，`.onnx` 只有 33 KB，需 `external_data=False`；② 新导出器要求 `opset ≥ 18` 且需要 `onnxscript`
额外收获	导出时自动发生 **BN 折叠**（19 个节点里一个 BatchNormalization 都没有），且精度无损

✅ 第 4 课：延迟与内存基准测试

学什么	warmup、p50/p95、吞吐 vs 延迟、峰值显存、线程数影响；**GPU 计时为什么必须用 CUDA Event**
动手	benchmark 脚本：4 个后端 × 4 种 batch，测延迟分布、吞吐、单张成本、峰值显存
验收	batch=1 实测 p50：**ONNX CPU 0.413 ms** vs PyTorch GPU 0.462 ms；GPU 在 batch=32 吞吐 **37915 张/秒**（13 倍于 CPU）

简历上能写	"建立推理基准测试流程，量化 4 个后端 × 4 种 batch 的延迟分布与吞吐；通过 ONNX Runtime 将 CPU 延迟从 1.37 ms 降至 0.41 ms（3.3 倍）"
核心发现	**GPU 在 batch=1 时并不比 CPU 快**（启动开销摊不开）——而边缘就是 batch=1，所以边缘主力是 CPU/NPU
其他发现	CPU 吞吐在 batch=8 就饱和（瓶颈是内存带宽不是算力）；单线程仅比多线程慢 2.5 倍（12 核 ≠ 12 倍）

⏸ 第 5 课：部署到真实目标硬件（待你提供设备，暂时跳过）

学什么	按平台选运行时：TensorRT(NVIDIA) / TFLite(Android) / NCNN(手机) / RKNN(Rockchip) / OpenVINO(Intel) / ONNX Runtime(通用)
动手	在你能拿到的真实设备上跑通推理并测延迟
验收	同一模型在 PC 与目标设备上的延迟对比表
坑点	算子不支持、输入预处理不一致、线程数配置

阶段三：模型压缩 ← 边缘 AI 最核心的技能

✅ 第 6 课：PTQ 训练后量化（INT8）

学什么	量化仿射映射（scale/zero_point）；权重 vs 激活；静态 vs 动态量化；**校准集只能用训练集**
动手	对第 3 课的 ONNX 模型做静态 PTQ + 动态量化对比 + 校准集大小消融
验收	实测 **8.412 MB -> 2.144 MB（3.92 倍）**，p50 **0.392 -> 0.125 ms（3.14 倍）**，准确率 **0.9215 -> 0.9180（-0.35 个百分点）**
简历上能写	"PTQ 静态量化将模型压缩 3.92 倍、CPU 延迟降低 3.14 倍，精度仅损失 0.35 个百分点"
⚠️ 重要反例	动态量化在卷积网络上导致 **24 倍性能退化**（0.392 -> 9.469 ms），根因是 `ConvInteger` 缺少优化内核且引入 16 个 Mul/8 个 Add 做运行时反量化

✅ 第 7 课：QAT 量化感知训练

学什么	假量化（Fake Quant）；**直通估计器 STE**；QAT 四步流程；什么时候该/不该上 QAT

动手	自己实现 FakeQuant + STE，插桩到卷积/全连接，微调 6 轮（110 秒）
验收	同口径实测：PTQ **-0.35pp** → QAT **-0.25pp**（净收益 **+0.10pp**），体积和延迟不变
诚实结论	**收益很小**（0.10pp）——因为 PTQ 本来就只掉 0.35pp。QAT 是"PTQ 掉太多时的补救"，不是默认选项
额外发现	QAT 会让 FP32 精度略降（0.9215 → 0.9205）：**它是一次权衡，不是免费提升**
方法论教训	差点把 2000 张和 10000 张测试集的数字混着比，得出"提升 0.52pp"的错误结论；写了 `final_compare.py` 统一口径后才修正为 0.10pp
简历上能写	"实现 Fake Quant + STE 的 QAT 流程，把 INT8 量化损失从 -0.35pp 改善到 -0.25pp；建立同口径对比脚本，修正跨测试集比较带来的 0.42pp 评估偏差"

✅ 第 8 课：剪枝与知识蒸馏

学什么	软标签/温度 T；KL 蒸馏损失；**结构化 vs 非结构化剪枝**；通道 L1 重要性；层间依赖
动手	① 从零训练 vs 蒸馏同架构 0.55M 学生；② 对 2.20M 老师做 50% 结构化剪枝 + 微调；③ 蒸馏学生再量化
验收	蒸馏 **+0.80pp**（0.8932 -> 0.9012，同参数同配置）；剪枝 **参数 -62.9%**、微调挽回 **66.19pp**（23.67% -> 89.86%）
最终成果	**蒸馏 + 量化：8.412 MB -> 0.565 MB（14.89 倍），精度仅 -1.80pp，延迟 6.69 倍**
⚠️ 重要结论	① 非结构化剪枝**不加速**；② 剪枝不微调会崩（92.18% -> 23.67%）；③ 本任务上**剪枝是三种手段里最差的**，重新设计小模型 + 蒸馏更划算
简历上能写	"组合蒸馏与 INT8 量化，将模型从 8.41 MB 压至 0.565 MB（14.89 倍），精度仅降 1.80pp，CPU 延迟降低 6.69 倍"

阶段四：数据与微调

✅ 第 9 课：做你自己的数据集

学什么	**数据泄漏**；**域偏移**；按实体划分（人/设备/时间/地点/场景）；类别不均衡；噪声底线
动手	构造 3000 场景 × 2 采集条件的数据集，对比三种划分策略在同一现场测试集上的表现

验收	实测：随机划分泄漏 **96%** 测试场景；仅清晰图训练在现场测试集上只有 **49.50%**（二分类瞎猜水平）；换成覆盖部署分布后 **61.17%（+11.67pp）**
附带发现	同配置两次运行的差异达 **3.2pp** —— 说明本实验噪声底线约 ±3pp，小于此的差异不可信
简历上能写	"定量验证数据泄漏与域偏移的影响：仅实验室数据训练的模型在现场测试集上仅 49.5%，引入现场数据后提升至 61.2%（+11.7pp）"

✅ 第 10 课：迁移学习与微调

学什么	预训练特征为什么可迁移；冻结 vs 微调；分层学习率；**灾难性遗忘**；域差异如何决定策略
动手	同一架构（MobileNetV3-Small）6 组对照：从零 / 冻结 / 微调×2 学习率 / 80 张数据×2 策略
验收	实测：400 张数据下从零训练 **49.50%**（=瞎猜）→ 微调 **62.67%（+13.2pp）**
⚠️ 诚实记录	**两条教科书最佳实践在本任务失效**：① 微调 lr=0.01 比 0.001 高 3.83pp；② 80 张数据时微调仍比冻结高 5.33pp。定位原因为域差异过大 + 训练轮数不足
简历上能写	"验证出小学习率与冻结 backbone 两条通用实践在本任务失效，定位为预训练域与目标域差异过大，并给出按域差异选策略的决策建议"

✅ 第 11 课：端到端 pipeline 与作品集

学什么	把 1–10 课串成一条命令；实验可复现；结果记录
动手	一条命令完成 训练→导出→量化→基准测试→出报告
验收	别人拿到你的仓库能一键复现
简历上能写	这个仓库本身就是简历项目

进度

课	主题	状态
1	训练循环的五件事（MLP）	✅ test acc **53.33%**
2	从 MLP 到 CNN，做到 90%	✅ test acc **92.18%**

课	主题	状态
3	导出 ONNX + 推理 + 对齐	✅ 对齐误差 **2.03e-06**，单文件 **8.41 MB**
4	延迟与内存基准测试	✅ 实测 4 后端 × 4 batch，找到"GPU batch=1 不占优"
5	部署到真实目标硬件	⏸ **待硬件**（可跳过）
6	PTQ 量化（INT8）	✅ **已完成**，体积 3.92x↓ / 延迟 3.14x↓ / 精度 -0.35pp
7	QAT 量化感知训练	✅ **已完成**，量化损失 -0.35pp → **-0.25pp**
8	剪枝与知识蒸馏	✅ **已完成**，蒸馏+量化 **14.89x 压缩 / -1.80pp**
9	做你自己的数据集	✅ **已完成**，域偏移代价 **49.5% → 61.2%**
10	迁移学习与微调	✅ **已完成**，预训练 **+13.2pp**；两条最佳实践失效
11	端到端 pipeline 与作品集	✅ **已完成**，一条命令 288 秒跑完全流程

学完之后的简历项目描述（模板）

端侧图像分类模型的全链路压缩与部署
- 基于 PyTorch 训练 CIFAR-10 分类网络，测试准确率 **92.2%**
- 导出为 **ONNX 单文件**并完成数值对齐验证（最大误差 <1e-5），
建立 **ONNX Runtime 基准测试流程**，量化 p50/p95 延迟与峰值内存
- 通过 **PTQ/QAT 量化**将模型从 FP32 压缩至 INT8，**体积减少 4 倍**，
推理延迟从 __ **ms 降至** __ **ms**，精度损失 **< 1%**
- 使用 **知识蒸馏** 将参数量从 __**M 降至** __**M**，进一步降低端侧内存占用
- 产出可复现的端到端 pipeline（训练→导出→量化→基准→报告）

面试高频问题（第 11 课后你应该能全部答上）

1. 训练框架和推理框架的区别？为什么设备上不能装 PyTorch?
2. 导出 ONNX 后你怎么确认模型没坏?
3. **导出后文件只有 33 KB 而模型应该 8 MB，你会怎么排查?**
4. 为什么测延迟要先 warmup？为什么看 p95 而不是只看平均?
5. PTQ 和 QAT 的区别？什么时候必须用 QAT?
6. 量化后精度掉了 5%，你怎么排查?

7. 为什么非结构化剪枝通常不能加速?
8. 模型 2.2M 参数，FP32 和 INT8 分别占多少内存?
9. 你的验证集怎么划分的？为什么不能用公开测试集选模型?
10. 数据只有 500 张，你怎么训?

与"从零训 GPT"那套的关系

之前做的	在边缘 AI 里的位置
预训练（15000 步）	❌ 用不到，了解概念即可
SFT 指令微调	✅ 就是第 10 课（微调）的同类工作
从零写 Transformer	❌ 用不到
参数量 ↔ 体积换算	✅ 边缘第一课就要会算

结论：之前那个项目最大的价值是让你见过完整的训练循环和微调流程，
而不是"学会了预训练"。

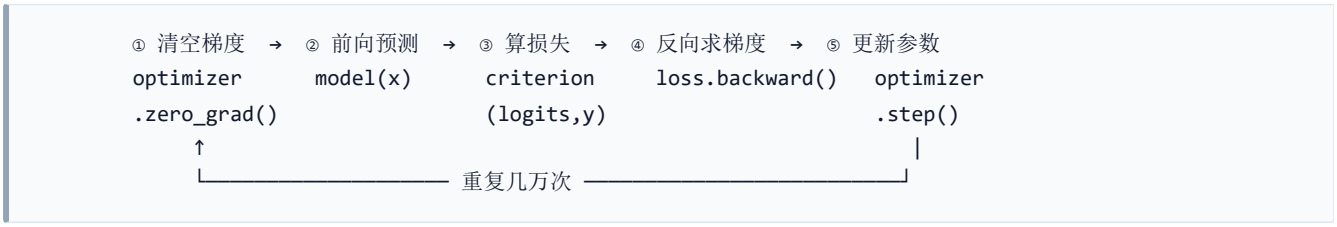
第 1 课

训练循环的五件事

> **本课目标**：搞懂 PyTorch 训练一个模型时到底发生了什么。
> **不要追求准确率**——本课用的是最笨的 MLP，50% 是正常的。

一、先记住这张图

任何深度学习训练，本质上就是这个循环：



就这五步，没有别的。 所有深度学习框架、所有模型、所有论文，底层都跑这个循环。
我们的 `train_mlp.py` 里，这五步被框起来了，去代码里找：

```
# ============ 训练循环的 5 步，背下来 ============
optimizer.zero_grad()            # 1. 清空上一轮累积的梯度
logits = model(x)                # 2. 前向：算出预测
loss = criterion(logits, y)      # 3. 算损失：错了多少
loss.backward()                  # 4. 反向：autograd 自动算出每个参数的梯度
optimizer.step()                 # 5. 更新：按梯度方向微调参数
# ================================================
```

二、五个核心概念

[概念 1] Tensor —— 数据的基本单位

Tensor 就是**多维数组**，和 NumPy 的 `ndarray` 几乎一样，但多了两个能力：

1. **能在 GPU 上算**（NumPy 只能在 CPU）
2. **能记住自己是怎么算出来的**（这是自动求导的基础）

形状约定（务必记住，边缘 AI 天天用）：

数据	形状	含义
一批图片	`(B, 3, 32, 32)`	B=批大小, 3=通道, 32×32=高宽
一批标签	`(B,)`	每张图一个整数类别
模型输出	`(B, 10)`	每张图对 10 个类别的"分数"

注意 CPU/GPU 搬运：数据默认在 CPU，模型和数据必须在**同一个设备**上，否则报错。`x.to(device)` 就是在搬数据。这是新手最常见的报错来源之一。

[概念 2] Dataset / DataLoader —— 怎么把数据一批批喂进去

你不可能把 5 万张图一次性塞进显存。所以要**分批**。

- **Dataset**：回答"第 i 个样本长什么样"。包含数据和标签。
- **DataLoader**：负责切 batch、打乱顺序、多进程预取。

```
for x, y in train_loader:      # 每次循环拿到一批（batch_size 张）
    ...
```

为什么要打乱（shuffle=True）？ 如果按顺序喂，模型会先学完"飞机"再学"汽车"，学后面的会把前面的忘掉。打乱能让每批数据都混合各类别。

为什么测试集不打乱？ 因为测试只看总准确率，顺序无关，打乱反而浪费时间。

[概念 3] nn.Module —— 模型长什么样

模型就是一个"可调参数的函数"。你只需要定义**前向传播**（输入怎么变成输出）：

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(...)   # 声明有哪些层（以及它们的参数）

    def forward(self, x):
        return self.net(x)              # 定义数据怎么流过这些层
```

反向传播不用你写。 PyTorch 会根据 `forward` 里用到的运算自动推导出反向过程。这是框架最核心的价值。

[概念 4] loss + autograd —— 怎么算"错了多少"，以及"谁该负责"

Loss（损失函数）：把"预测"和"真值"的差距压缩成**一个数字**。
分类任务用 `nn.CrossEntropyLoss`。

- loss 大 → 模型很差
- loss 小 → 模型很好
- **训练的全部目标就是让这个数字变小**

autograd（自动求导）：`loss.backward()` 这一行，PyTorch 会自动算出**每一个参数**对 loss 的"责任大小"（梯度）。

> 直观理解：梯度告诉你"这个参数往哪个方向调、调多少，loss 会下降"。
> 有几百万个参数，你不可能手算——这就是深度学习框架存在的意义。

[概念 5] optimizer —— 怎么改参数

知道梯度还不够，还要决定"每次改多少"。这就是优化器。

优化器	特点
SGD	最朴素，需要仔细调学习率
SGD + Momentum	加惯性，收敛更快
Adam / AdamW	自适应学习率，**默认首选**，能省很多调参功夫

`lr`（学习率）是最重要的超参数：

- **太大**：loss 剧烈震荡甚至变成 `nan`（模型"学飞了"）
- **太小**：loss 下降极慢，时间全浪费

三、动手

```
cd edge-ai\lesson01
& E:\llm-env\venv\Scripts\python.exe train_mlp.py --epochs 10
```

预计：10 个 epoch，**测试准确率 50% 左右**，每轮几秒钟。

> 50% 不是失败。CIFAR-10 的 MLP 本来就只能到 50% 上下——
> 因为它把图片**拉平**了，丢失了"相邻像素有关联"这个信息。
> 学 CNN 就是为了解决这个问题（第 2 课）。

四、作业（必须动手，不做等于没学）

作业 1：观察学习率的作用

分别用 `--lr 0.1`、`--lr 0.001`、`--lr 0.00001` 跑 3 次。

> **你应该观察到**：0.1 的 loss 会震荡甚至 nan；0.00001 的 loss 几乎不下降。
> **这就是学习率最重要的证据——亲手做一遍比看十篇文章有用。**

作业 2：观察 batch size 的取舍

分别用 `--batch_size 16` 和 `--batch_size 1024`。

> **你应该观察到**：小 batch 每轮更慢（循环次数多），但往往精度略高；
> 大 batch 吞吐高，但可能需要调大学习率。
> **边缘 AI 视角**：这是"训练时间 vs 精度"的第一次取舍。

作业 3：算模型体积

代码里已经打印了参数量和 FP32 体积。现在算：如果量化成 INT8，体积是多少？

> **答案**：参数量 × 1 字节。这就是第 7 课量化的核心——
> **INT8 相比 FP32 直接省 4 倍内存**，这是边缘 AI 最基础的一课。

作业 4（进阶）：动手改模型

把 `nn.Linear(in_dim, 512)` 和 `nn.Linear(512, 256)` 改成 `128` 和 `64`。

> **你应该观察到**：参数量大幅下降，准确率也下降。
> 记下这两个数字——第 9 课"剪枝/蒸馏"要的就是**在精度损失很小的情况下把参数砍掉。**

作业 5（进阶）：加 Dropout 看差距

在两层之间加 `nn.Dropout(0.3)`。

> **你应该观察到**：train acc 下降，但 train 和 test 的差距缩小。
> 这就是"正则化"的作用——**抑制过拟合**（第 3 课的主题）。

五、验收标准

全部答得上来，才算过关：

- [ ] 不看代码，说出训练循环的 5 步
- [ ] 说出 `optimizer.zero_grad()` 为什么必须存在（提示：不清空会怎样？）
- [ ] 说出 `model.train()` 和 `model.eval()` 的区别，以及为什么需要它
- [ ] 说出 `with torch.no_grad()` 的作用和好处
- [ ] 给定"10M 参数"，算出 FP32 和 INT8 各占多少 MB
- [ ] 解释为什么 MLP 在 CIFAR-10 上只有 50%

六、自测答案

▶ **点开对答案**

七、下一课预告

第 2 课：从 MLP 到 CNN，把准确率做到 90%。
你会学到卷积为什么有效、BatchNorm 解决什么问题、数据增强为什么能提升泛化。

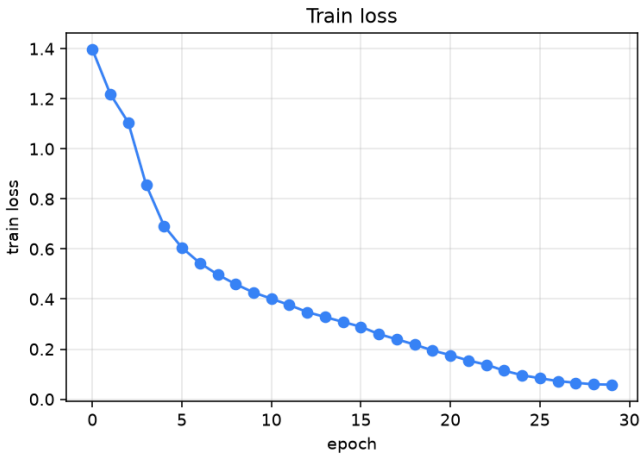
完成本课后，在 `ROADMAP.md` 里给第 1 课打勾。

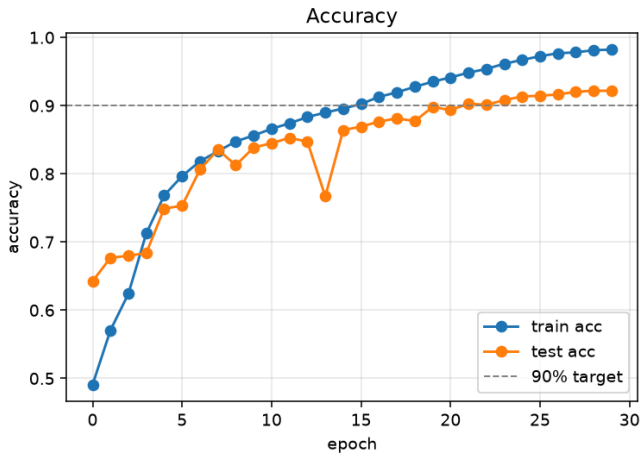
第 2 课

从 MLP 到 CNN

本课目标：搞懂卷积网络为什么能看懂图像，并把 CIFAR-10 准确率做到 90% 以上。
本课还有一个**真实的调试案例**——模型完全不学，我们一步步把它揪出来。

一、结果





指标	第 1 课 MLP	第 2 课 CNN
测试准确率	53.3%	**92.18%**
参数量	1.71 M	2.20 M
FP32 体积	6.51 MB	8.38 MB
INT8 体积	1.63 MB	2.10 MB
每轮耗时	10.2 s	14.4 s
train / test 裂口	74% / 53%（**裂口 21%**）	98% / 92%（**裂口 6%**）

两个数字一起看：准确率从 53% 涨到 92%，同时 train/test 的裂口从 21% 缩小到 6%。
后者说明 CNN 学到的是**可泛化的规律**，而 MLP 是在死记硬背。

二、为什么 MLP 不行、CNN 行

	MLP	CNN
处理方式	`Flatten`：32×32×3 → 一根 3072 维的线	3×3 卷积核在图上滑动
空间结构	**彻底打散**——第 (0,0) 个像素和第 (0,1) 个像素被当成两个无关的特征	**保留**——卷积核只在局部窗口内做加权
平移不变性	猫在左上角和右下角，对 MLP 是两组完全不同的输入	同一个卷积核滑过全图，猫在哪都认得出来
参数量	与图像大小成正比	**与图像大小无关**（只和卷积核大小、通道数有关）

> 一句话：**卷积把"图像是二维的、相邻像素有关联"这个先验知识写进了网络结构里。**
> 深度学习的大部分进步，都是"往结构里塞正确的先验"。

三、六个新概念

[新概念 1] 卷积层 `nn.Conv2d`

64 个 3×3 的卷积核在图上滑动，每个位置做一次加权求和。

- **参数量** = 输入通道 × 输出通道 × 核高 × 核宽 = `3 × 64 × 3 × 3 = 1728`
- **关键**：这个数字**与图像大小无关**。输入 32×32 还是 224×224，参数量都是 1728
- `padding=1` 让输出保持 32×32（否则每卷一次边缘就少 1 像素）
- `bias=False` 是因为后面紧跟 BatchNorm，BN 自带偏置

[新概念 2] 池化 `nn.MaxPool2d`

每 2×2 区域取最大值，高宽各减半：`32 → 16 → 8 → 4`。

作用有两个：① 减少后面的计算量；② 让每个神经元"看到"更大的范围（**扩大感受野**）。

[新概念 3] BatchNorm

把每层的输出在 batch 维度上标准化（减均值、除标准差），再学一个缩放和平移。

为什么必须有它：没有 BN，这么深的网络很难训得动。BN 让每一层的输入分布保持稳定，使得较大的学习率可以用、收敛速度快很多。

> **代价**：BN 在推理时要用训练时统计的 `running_mean/running_var`。参数翻倍、多一次运算——边缘部署时 BN 通常会被**折叠进前面的卷积**（第 7 课会讲）。

[新概念 4] 数据增强（边缘 AI 里最划算的手段）

训练时每次取图都随机变一下：

```
T.RandomCrop(32, padding=4)   # 四周补 4 像素再随机裁回 32×32 -> 物体位置变化
T.RandomHorizontalFlip()      # 随机左右翻转 -> 镜像的猫还是猫
```

同一张图每个 epoch 看到的都不一样，**等效于把数据集变大了很多倍**，而代价几乎为零。

> 这就是为什么它对本课特别重要：train/test 裂口从 21% 缩到 6%，主要功臣就是它（加上 BN 和 Dropout）。**在边缘 AI 里，你没有钱买几十万张标注数据，数据增强是你最便宜的"扩充数据集"手段。**

[新概念 5] 学习率调度：预热 + 余弦退火

```
前 2 轮：线性预热，lr 从 0.005 升到 0.05
之后   ：余弦退火，lr 从 0.05 平滑降到 0
```

- **预热**：刚初始化时 BatchNorm 的统计量还不准，此时用大学习率会把网络打飞
- **余弦退火**：后期学习率变小，模型能稳定收敛到最后一点精度

本课就是栽在预热上的，见下一节。

[新概念 6] Dropout

训练时随机把一部分神经元的输出置 0（这里是 40%），强迫网络不要依赖某几个特定神经元。

注意：**评估时 Dropout 自动关闭**——这就是 `model.eval()` 的作用之一。

四、【本课重点】一次真实的调试：模型完全不学

现象

第一次跑 30 轮，结果是：

```
epoch  0 | loss 2.3406 | train 0.104 | test 0.100
epoch  5 | loss 2.3042 | train 0.099 | test 0.100
epoch 16 | loss 2.3034 | train 0.098 | test 0.100
```

loss 死死卡在 2.3043 不动，准确率永远 10%。

> `2.3026 = ln(10)`。这是 10 类分类"完全瞎猜"时的理论损失值。
> **记住这个数字**：以后看到 loss 卡在 `ln(类别数)`，就说明模型根本没在学习，
> 而不是"效果不好"。这两件事要分清楚——前者是 bug，后者是调优。

排查过程（`python diagnose.py` 可复现）

按顺序检查 4 件事：

检查	结果	判断
1. 数据	范围 [-1.000, 1.000]，均值 -0.213，标准差 0.546	✅ 正常
2. 模型初始输出	logits 标准差 0.37	✅ 正常（不是恒为 0）
3. 第一次反向的梯度	总梯度范数 **5.61**	✅ 正常（不是梯度消失或爆炸）
4. 换优化器/学习率跑 40 步	见下表	❌ **找到问题了**

第 4 步的结果：

优化器	40 步后 loss	降幅	判断
SGD lr=0.10 无预热	2.3998 → 2.3013	0.0985	❌ **几乎没学**
SGD lr=0.05 无预热	2.3998 → 1.8754	0.5244	⚠️ 勉强
SGD lr=0.01 无预热	2.3998 → **1.6093**	**0.7905**	✅ **反而最好**
Adam lr=0.001	2.3998 → 1.9912	0.4086	⚠️ 一般

结论

学习率 0.10 对这个网络来说太大了。

反直觉的点在于：`lr=0.01` 反而比 `lr=0.10` 学得好那么多。
说明 0.10 不是"学得慢"，而是**最初几步就把网络震荡出去了**——一旦输出变成常数，
梯度就再也推不动它，网络永久卡死在瞎猜状态。

修复

不是把学习率一路调小，而是**加学习率预热**：

```
warmup = torch.optim.lr_scheduler.LinearLR(optimizer, start_factor=0.1, total_iters=2)
cosine = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=28)
scheduler = torch.optim.lr_scheduler.SequentialLR(
    optimizer, schedulers=[warmup, cosine], milestones=[2])
```

加上预热后：

配置	6 轮测试准确率	30 轮测试准确率
lr=0.02 + 预热	84.1%	—
lr=0.05 + 预热	83.5%	**92.18%**
lr=0.10 + 预热	83.8%	—

这次调试教给你的事

1. `loss = ln(类别数)` **是一个明确的信号**：模型没在学，去找 bug，别去调参。
2. **排查顺序是固定的**：数据 → 初始输出 → 梯度 → 优化器。
   从最底层往上层查，别一上来就怀疑模型结构。
3. **"学习率太大会不学"是反直觉但真实存在的。**直觉上 lr 越大应该越快，
   实际上一旦把网络打飞，它再也回不来。
4. **同一个现象可能有多种原因。**如果不做第 4 步的对照实验，你很可能会去改模型结构、
   换数据集、加正则——全是白费功夫。**用对照实验定位，不要靠猜。**

> 这个排查流程（`diagnose.py`）在之后每一课都会用得上。
> 到了第 7 课量化精度掉太多时，你会用到一模一样的思路。

五、作业

作业 1：看看数据增强到底值多少精度

```
& $py train_cnn.py --epochs 15 --no_augment
& $py train_cnn.py --epochs 15
```

> **你应该观察到**：关掉增强后 train acc 冲得更高，但 test acc 更低，裂口明显变大。
> **把两个 test acc 记下来**——这个差值就是数据增强的价值。

作业 2：亲手复现"模型不学"的 bug

把 `train_cnn.py` 里的 `--warmup_epochs` 改成 0，跑 3 轮。

> **你应该观察到**：loss 迅速卡到 2.30 附近，准确率 10%。
> **然后跑** `python diagnose.py`，看懂那 4 步检查。
> **这一条最重要**——亲手制造一次 bug 并定位它，比看十篇文章有用。

作业 3：学习率对照实验

用 `--epochs 6` 分别跑 `--lr 0.02` / `0.05` / `0.10` / `0.20`。

> **你应该观察到**：0.20 开始不稳定或变差；0.02 起步慢但稳。
> **这就是"调参"的真实样子——做对照实验，不是靠感觉。**

作业 4：把模型变窄，体会边缘 AI 的取舍

把 `widths` 从 `(64, 128, 256)` 改成 `(32, 64, 128)`。

> **你应该观察到**：参数量降到约 1/4，精度掉几个点，训练快很多。
> **把「参数量 / 精度 / 耗时」三个数记成一张表**——这就是边缘 AI 每天都在做的权衡。
> （第 9 课的剪枝和蒸馏，本质就是在做这件事。）

作业 5：算清楚部署成本

用打印出来的参数量，算：
- INT8 量化后模型多大?
- 如果目标设备只有 2 MB 的模型存储空间，这个模型能放下吗?
- 如果要放进去，需要把参数量压到多少以下?

六、验收标准

- [ ] 能解释卷积为什么比全连接省参数

- [ ] 能说出 BatchNorm 解决了什么问题，以及它在推理时有什么代价
- [ ] **看到 loss 卡在 ln(类别数)，知道这是 bug 而不是效果问题**
- [ ] 能背出排查"模型不学"的 4 个步骤
- [ ] 能解释为什么学习率太大会导致完全不学
- [ ] 能说出数据增强为什么能提升泛化
- [ ] 给定参数量，能算出 FP32 / INT8 的模型体积

七、下一课预告

第 3 课：训练诊断与实验管理。
你会学会读 loss 曲线的每一种形状、系统地做对照实验、以及如何让实验可复现。

然后进入整个路线的**分水岭**：第 4 课把模型导出成 ONNX，真正开始"部署"。

完成本课后，在 `ROADMAP.md` 里给第 2 课打勾。

第 3 课

导出 ONNX

> **这是整个路线的分水岭。**
> 第 1-2 课的产出是一个"PyTorch 模型"，只能在装了 PyTorch 的机器上跑。
> 从这一课开始，产出变成一个**能部署的模型文件**。

一、结果速览

指标	值
导出文件	`out/cnn.onnx`，**8.41 MB**（单文件、自包含）
opset	18
计算图节点数	**19** 个
PyTorch vs ONNX 最大输出误差	**2.03e-06** ✅
2000 张测试图准确率	PyTorch **0.9215** / ONNX **0.9215**
两者预测一致率	**100.00%** ✅
CPU 推理延迟（batch=1）	p50 **0.404 ms**，p95 **0.534 ms**

二、为什么不能直接把 PyTorch 塞进设备?

	PyTorch（训练框架）	ONNX Runtime（推理框架）
定位	让你**训练**模型	让你**跑**模型
计算图	动态图，每次前向都重新解析	静态图，一次解析反复执行
依赖	Python 解释器 + 几百 MB 的库	一个 C++ 运行库，几 MB
体积	装完 torch ≈ 4 GB	几百 KB ~ 几 MB
目标硬件	服务器/PC 的 NVIDIA GPU	CPU / 手机 / NPU / 单片机 都能跑

你的边缘设备上没有 Python，也不可能装 4 GB 的 PyTorch。
所以必须把模型"翻译"成一种通用的中间格式，这正是 ONNX 做的事。

ONNX 是什么

> **ONNX = Open Neural Network Exchange**，模型界的"通用中间格式"。

打一个比方：

- PyTorch 模型 = 一份**中文写的菜谱**
- 目标设备 = 一个**只懂英文的厨师**
- ONNX = **标准化的英文菜谱格式**
- ONNX Runtime / TensorRT / TFLite = 各个厨师（**不同的执行后端**）

好处是：**导出一次，多个运行时都能读**。以后你要换硬件，不用重新训练、不用改模型，只要换一个运行时就行（第 5 课会看到这一点）。

三、动手：三步导出

```
cd edge-ai\lesson03
& E:\llm-env\venv\Scripts\python.exe export_onnx.py
```

代码里的三个关键点：

```
model.eval()                    # ① 必须！否则 Dropout/BN 行为不对，导出结果会错
dummy = torch.randn(1, 3, 32, 32)   # ② 给一个"示例输入"，导出器靠它追踪计算图

torch.onnx.export(
    model, dummy, "out/cnn.onnx",
    input_names=["input"], output_names=["logits"],
    opset_version=18,
    dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},  # ③ 动态 batch
    external_data=False,        # ④ 权重内嵌，产出单个文件
)
```

关于动态轴：不写 `dynamic_axes`，导出的模型只能接受 batch=1；
写了之后随便喂多少张都行。实测：

batch	ONNX 输出形状	与 PyTorch 的误差
1	(1, 10)	2.21e-06

batch	ONNX 输出形状	与 PyTorch 的误差
8	(8, 10)	2.38e-06
64	(64, 10)	3.22e-06

关于 `opset_version`：torch 2.11 起默认导出器换成了 `torch.export` 版本，它**要求 opset ≥ 18**，并且需要额外安装 `onnxscript` 包。装不对会直接报 `ModuleNotFoundError: No module named 'onnxscript'`。

四、【重点】导出后必须检查的两件事

导出"成功"不等于导出"正确"。这两项检查缺一不可。

检查 1：输出对齐（数值一致性）

做法：同一个输入，分别喂给 PyTorch 和 ONNX，比较输出。

```
torch_out = model(x).numpy()
onnx_out  = sess.run(["logits"], {"input": x.numpy()})[0]
max_diff = np.abs(torch_out - onnx_out).max()
```

本课实测：2.03e-06，判定 PASS（阈值 1e-4）。

> 为什么必须做？因为导出涉及**算子翻译**。某个算子翻译错了、opset 太低、忘了 `model.eval()`，都会导致精度悄悄下降。
> **如果不做这一步，你可能部署了一个 70% 准确率的模型，还以为它是 92%。**

我还额外做了一层检查——**在 2000 张真实测试图上对比准确率**：

```
PyTorch 准确率 0.9215
ONNX    准确率 0.9215
两者预测一致率 1.0000
```

这比逐个比较数值更有说服力：**2000 张图里，两个框架的预测一个都没差。**

检查 2：文件大小 ← 我们真的在这里踩了坑

第一次导出，日志显示：

```
大小: 0.03 MB   opset: 18   计算图节点数: 19
```

0.03 MB？模型明明是 8.38 MB。

查目录才发现：

```
cnn.onnx          0.0328 MB   <- 只有计算图
cnn.onnx.data     8.4375 MB   <- 权重跑这里了
```

torch 2.11 的新导出器**默认把权重写到外部文件**（ONNX 的 external data 格式）。

为什么这在边缘部署里是个问题？

- 很多推理运行时**读不了外部数据文件**
- 拷贝模型时容易只拷了 `.onnx` 漏了 `.data`，到设备上才报错
- 分发模型时你要管两个文件而不是一个

修复：加一句 `external_data=False`，权重就内嵌了：

```
大小: 8.41 MB   opset: 18   计算图节点数: 19
```

我在脚本里加了一道自检：

```
if size_mb < expect_mb * 0.5:
    print("!! 警告: ONNX 只有 %.2f MB，但权重应该有 %.2f MB" % (size_mb, expect_mb))
```

> **教训：导出后永远看一眼文件大小。** 这是最便宜、最容易漏的一项检查。

顺便说一句诚实的话

ONNX 的 8.41 MB **比 PyTorch 的 8.38 MB 还大一点点。**

ONNX 不是压缩格式，它只是换了个格式。想要模型变小，得靠**第 6 课的量化。**
不要把"导出 ONNX"当成"模型压缩"——这是新手常见的误解。

五、意外收获：导出时自动发生的算子融合

看导出后的计算图：

```
算子构成: Relu×7, Conv×6, MaxPool×3, Gemm×2, Reshape×1   (共 19 个节点)
```

对比一下 PyTorch 里的结构：8 个 `conv_bn_relu` 块 = **6 个 Conv + 6 个 BatchNorm + 6 个 ReLU**，再加分类头。

但 ONNX 图里**一个 BatchNormalization 节点都没有**！

原因：推理时 BatchNorm 是一个固定的线性变换（用训练时统计的 `running_mean` / `running_var`），可以**在数学上等价地折叠进前面的卷积权重**里。
这叫 **BN 折叠（BN folding）**。

同时 `Dropout` 也消失了——它在推理时本来就是个空操作。

这是导出带来的免费优化：算子变少 → 推理更快，而且精度完全不变（误差 2e-6）。

> 记住这个概念。第 6 课量化时，你会看到量化工具**依赖**这种融合后的图结构：
> 带 BN 的图和折叠后的图，量化效果不一样。

六、延迟测量与 warmup

实测（CPU，batch=1，warmup 20 次后测 200 次）：

指标	值
平均	0.420 ms
p50	**0.404 ms**
p95	**0.534 ms**
最小	0.349 ms

为什么要 warmup?

第一次推理特别慢，因为运行时要做一堆一次性的事情：

1. 分配内存池
2. 为每个算子选择最优实现（比如选哪个卷积算法）
3. 预热 CPU 缓存

如果你不 warmup 就直接计时，测到的是"启动开销"而不是"稳态性能"，
结果会**严重偏高**。所以标准做法是：**先空跑几十次，再开始计时。**

为什么要看 p95 而不是只看平均?

边缘 AI 里，**卡顿比慢更致命。**

自动驾驶、工业质检这类场景关心的是"最坏情况有多慢"。p95 = 100 次里有 5 次比它慢。
本课 p95 (0.534) 比 p50 (0.404) 高 32%——这个差距在真实设备上可能大得多
（温升降频、内存争抢、其他进程干扰）。

> 第 4 课会系统做这件事：不同 batch、不同后端、内存占用、以及怎么测才可信。

七、作业

作业 1：亲手复现"33 KB 的坑"

把 `export_onnx.py` 里 `external_data=False` 那行注释掉，重新导出。

> **你应该观察到**：`.onnx` 只有几十 KB，旁边多出一个 `.onnx.data`。
> 然后打开 `out/` 目录自己确认一遍——**这个坑值得亲手踩一次。**

作业 2：验证"算子融合"是真的

用 netron（https://netron.app，把 .onnx 拖进去）打开 `cnn.onnx`。

> **你应该观察到**：图里找不到 BatchNormalization 节点，也找不到 Dropout。
> 把鼠标放到 Conv 节点上看权重形状——**BN 已经被折进去了。**

作业 3：测出动态轴的价值

写一小段代码，用 ONNX Runtime 分别推理 batch=1 和 batch=64，各测 100 次。

> **你应该观察到**：batch=64 的单次延迟远大于 batch=1，但**摊到每张图上更便宜。**
> 这就是"吞吐 vs 延迟"的取舍，第 4 课会详细算。

作业 4：算一算部署账

现在你手上有三个数字：`cnn.onnx` 是 8.41 MB、p50 延迟 0.404 ms、准确率 92.15%。

假设你的目标设备是：
- 存储空间只剩 **4 MB**
- 单次推理预算 **10 ms**

> **问题**：这个模型能部署吗？如果不能，需要改什么？
> （答案：存储放不下——需要第 6 课的量化把 8.41 MB 压到约 2.1 MB；延迟倒是绰绰有余。）

八、验收标准

- [ ] 能说出训练框架和推理框架的区别，以及为什么设备上不能装 PyTorch
- [ ] 能解释 ONNX 的作用，以及"导出一次、多个运行时共用"的好处
- [ ] **知道导出后必须做哪两项检查**（数值对齐 + 文件大小）
- [ ] 能解释为什么必须 `model.eval()` 之后再导出
- [ ] 能说出 warmup 的作用，以及为什么看 p95 而不是只看平均
- [ ] **知道 ONNX 不是压缩格式**，压缩要靠量化
- [ ] 能说出 BN 折叠是什么、为什么推理时可以折叠

九、下一课预告

第 4 课：延迟与内存基准测试。
你会做出一个真正的 benchmark 工具：扫描不同 batch size、对比 PyTorch GPU 与 ONNX Runtime CPU、测峰值内存、画吞吐-延迟曲线。

然后第 5 课把它部署到真实目标硬件上。

完成本课后，在 `ROADMAP.md` 里给第 3 课打勾。

第 4 课

延迟与内存基准测试

本课目标：把"模型跑多快"从**感觉**变成**可信的数字**，
并且知道这些数字在不同场景下意味着什么。

一、结果总表

同一模型（2.20M 参数），四个后端 × 四个 batch。

batch = 1 —— 边缘场景的标准配置

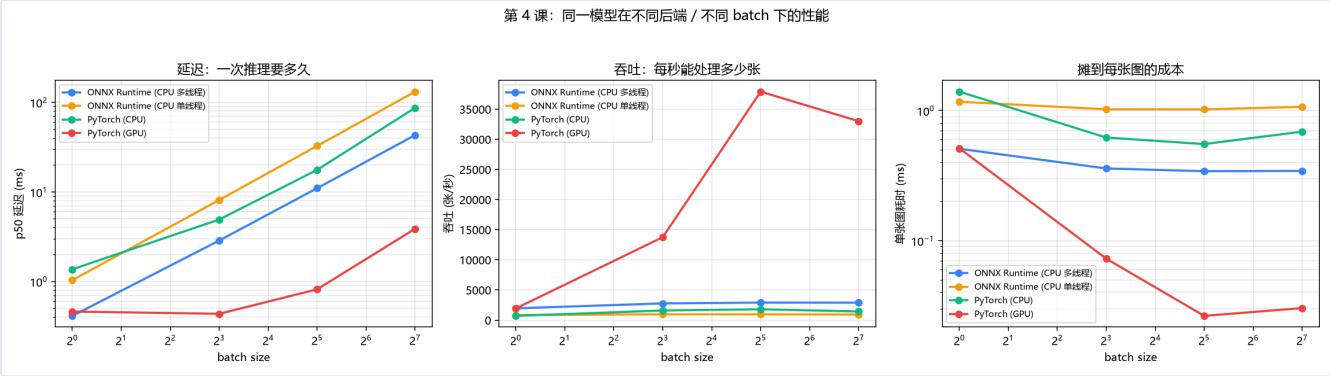
后端	p50 延迟	p95 延迟	吞吐	相对最快
ONNX Runtime (CPU 多线程)	**0.413 ms**	0.678 ms	1966 张/秒	**1.0x**
PyTorch (GPU, RTX 5060 Ti)	0.462 ms	0.828 ms	1950 张/秒	1.1x
ONNX Runtime (CPU 单线程)	1.034 ms	1.356 ms	854 张/秒	2.5x
PyTorch (CPU)	1.367 ms	1.788 ms	715 张/秒	3.3x

batch = 128 —— 云端/批处理场景

后端	p50 延迟	吞吐	单张成本
PyTorch (GPU)	3.873 ms	**33022 张/秒**	0.030 ms/张
ONNX Runtime (CPU)	43.036 ms	2908 张/秒	0.344 ms/张
PyTorch (CPU)	86.786 ms	1451 张/秒	0.689 ms/张
ONNX Runtime (CPU 单线程)	131.476 ms	934 张/秒	1.070 ms/张

GPU 峰值显存

batch	1	8	32	128
峰值显存	31.66 MB	32.11 MB	50.04 MB	147.16 MB



二、五个可以直接下结论的发现

发现 1（最重要）：GPU 在 batch=1 时并不比 CPU 快

```
ONNX Runtime CPU  p50 = 0.413 ms
PyTorch GPU       p50 = 0.462 ms     <- GPU 反而慢 12%
```

为什么？ GPU 快是因为它有几万个核心可以并行。但 batch=1 时只有一张图要算，这些核心大部分在闲着，**GPU 启动一次计算的开销（kernel launch）反而摊不开。**

但当 batch 涨到 32：

```
ONNX Runtime CPU  = 2923 张/秒
PyTorch GPU       = 37915 张/秒    <- GPU 快 13 倍
```

> **这就是边缘 AI 最核心的一课：**
> **边缘设备一次只处理一个请求（batch=1），用户就在旁边等着。**
> 所以边缘推理的主力是 **CPU / NPU**，而不是 GPU。
> 一张 8 GB 的显卡在边缘场景里可能是浪费——你付了钱，却用不上它的并行能力。

发现 2：换个推理框架，白赚 3.3 倍

同样在 CPU 上，同样 batch=1：

后端	p50 延迟
PyTorch	1.367 ms
ONNX Runtime	**0.413 ms**

代码没改、模型没改、硬件没改，只是换了个运行时，快了 3.3 倍。

为什么？ONNX Runtime 会做一批 PyTorch 不做的事：算子融合（第 3 课看到的 BN 折叠）、为每种算子预先挑选最优实现、内存池复用、针对 CPU 指令集（AVX-512）的手写内核。

> **简历上这就是一条实打实的优化成果**：
> "通过 ONNX Runtime 替换 PyTorch 推理，CPU 延迟从 1.37 ms 降至 0.41 ms（3.3 倍）"。

发现 3：吞吐在 batch=8 就饱和了 —— 瓶颈不是算力

看 ONNX Runtime CPU 的吞吐：

batch	1	8	32	128
吞吐(张/秒)	1966	2785	**2923**	2908

从 batch=8 到 batch=128，**吞吐基本没变**（2785 -> 2908，只涨 4%），
而延迟一路涨到 43 ms。

这说明 CPU 已经不是在"算"上受限，而是在"搬数据"上受限——内存带宽打满了。
再加大 batch 只是让队列变长、延迟变高，吞吐不会提升。

> **实用结论**：在这台机器上跑这个模型做推理服务，
> **batch 开到 8~16 就够了**，再大只会让延迟变差而吞吐不变。
> 这个"甜点位置"必须靠实测找，靠猜是猜不出来的。

发现 4：单线程只有多线程的 40%——但也没到 1/12

这台机器有 **12 个逻辑核**，但：

配置	p50 延迟	加速比
单线程	1.034 ms	1.0x
多线程（默认 12 核）	0.413 ms	**2.5x**

12 个核只换来 2.5 倍，因为和发现 3 同样的原因：内存带宽先饱和了。

> **对边缘的意义**：很多边缘设备只有 2-4 个核，甚至被其他进程占着。
> 你不能假设"12 核能带来 10 倍加速"。**部署前必须在目标设备上实测，**
> **并且试试限制线程数——有时候单线程反而更稳（p95 更低）。**

发现 5：batch 越小，抖动越大

后端 / batch	p50	p95	p95/p50
ONNX CPU / batch=1	0.413	0.678	**1.64x**
ONNX CPU / batch=128	43.036	46.807	1.09x
GPU / batch=1	0.462	0.828	**1.79x**

batch 越小，抖动越大——因为固定开销（kernel launch、函数调用、缓存未命中）在总时间里的占比更高。GPU 在 batch=1 时 p95 比 p50 高了 79%。

> **边缘场景恰恰就是 batch=1**，所以对边缘来说，**p95 比 p50 重要得多。**
> 只报平均值是在掩盖问题。

三、概念详解

[新概念 1] GPU 计时必须用 CUDA Event

这是新手做 GPU 性能测试**最容易犯的错**：

```
t0 = time.perf_counter()
model(x)                    # <- 这行几乎立刻返回！
t1 = time.perf_counter()    # <- 测到的是"排队时间"，不是"计算时间"
```

因为 PyTorch 的 GPU 调用是**异步**的：model(x) 只是把任务丢进 CUDA 队列就返回了，CPU 根本不知道 GPU 什么时候算完。

两种正确做法：

```
# 做法 A：同步等待（简单，但同步本身有开销）
torch.cuda.synchronize()
t0 = time.perf_counter()
model(x)
torch.cuda.synchronize()
t1 = time.perf_counter()

# 做法 B：CUDA Event（本课用的，最准）
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)
start.record()
model(x)
end.record()
torch.cuda.synchronize()
ms = start.elapsed_time(end)
```

做法 B 让 GPU 自己打时间戳，测的是纯计算时间，不包含 CPU 侧的排队开销。

[新概念 2] 延迟 vs 吞吐

	定义	谁关心
延迟	处理**一个**请求要多久	边缘设备、实时交互、自动驾驶
吞吐	**每秒**能处理多少个	云端服务、批量离线处理

两者经常**互相矛盾**：加大 batch 能提升吞吐（GPU 利用率高），但会**推高延迟**（要等一批凑齐才能开始算）。

> **边缘 AI 的第一原则：先确定你优化的是延迟还是吞吐。**
> 我们的场景（端侧单张图片推理）**是延迟**，所以 batch=1 就是唯一有意义的配置。

[新概念 3] 为什么必须看 p50 / p95

平均值会骗人。假设 100 次推理里 99 次是 1 ms、1 次是 500 ms：

- 平均值 = 5.99 ms，看起来还行
- 但实际上**有 1% 的用户等了半秒**

p95 = 100 次里有 5 次比它慢。它告诉你"最坏情况大概有多坏"。

边缘设备的抖动来源比服务器多得多：温升降频、内存争抢、其他进程、电池省电策略。**所以边缘的 p95/p50 比值通常远大于本课测到的 1.6 倍。**

[新概念 4] 峰值显存怎么测

```
torch.cuda.empty_cache()
torch.cuda.reset_peak_memory_stats()   # 必须重置！否则会带上之前的分配
with torch.no_grad():
    for _ in range(20):
        model(x)
torch.cuda.synchronize()
peak = torch.cuda.max_memory_allocated()   # 峰值，不是当前
```

结果很有意思：

batch	1	8	32	128
峰值显存	31.66 MB	32.11 MB	50.04 MB	147.16 MB

batch=1 时只用了 31.66 MB，而模型权重本身就有 8.38 MB。
多出来的部分是 CUDA 上下文、cuDNN 工作区、中间激活值。

> **边缘视角**：显存里除了模型，还有一大堆"看不见的开销"。
> 算部署内存预算时，**不能只算模型大小**，必须实测峰值。

[新概念 5] 线程数对 CPU 推理的影响

```
so = ort.SessionOptions()
so.intra_op_num_threads = 1     # 限制为单线程
so.inter_op_num_threads = 1
sess = ort.InferenceSession(path, so, providers=["CPUExecutionProvider"])
```

边缘设备往往只有 1-4 个核，而且可能被摄像头采集、解码等任务占着。
在开发机上用满 12 核测出来的延迟，拿到设备上会差好几倍。

本课实测：单线程 1.034 ms vs 12 线程 0.413 ms（2.5 倍）。
如果你要部署到 4 核设备上，**至少应该按单线程的数字做预算。**

四、作业

作业 1：亲手验证"GPU 在 batch=1 不占优"

分别跑 --batches 1 和 --batches 256，对比 CPU 与 GPU 的吞吐。

> **你应该观察到**：batch=1 时两者接近；batch=256 时 GPU 遥遥领先。
> **把这两个数字记下来**——面试被问到"边缘为什么用 CPU 不用 GPU"，答案就在这里。

作业 2：找到吞吐的"甜点"

单跑 ONNX Runtime CPU，batch 取 1, 2, 4, 8, 16, 32, 64。

> **你应该观察到**：吞吐在某个 batch 之后就不再涨了。
> **这个点就是内存带宽饱和的位置。** 写下来——这就是"调优"的真实样子。

作业 3：p95 的真实代价

把 --min_iters 从 50 提到 2000，看 p95/p50 的比值怎么变。

> **你应该观察到**：样本越多，p95 越接近真实的尾部行为。
> **讨论**：只跑 50 次得到的 p95 可信吗?

作业 4：算一笔部署账

假设目标设备是 **4 核 ARM CPU**，没有 GPU，内存 512 MB。

1. 用哪个后端的数字做预算？（提示：看单线程那一行）
2. batch=1 的延迟是多少？如果要求 30 FPS（每帧 33 ms），够用吗?
3. 模型占多少内存？加上运行时开销后还剩多少余量?

> **参考答案**：单线程 p50 约 1.03 ms，离 33 ms 预算绰绰有余；
> 模型 8.38 MB，即使算上 3 倍运行时开销也远小于 512 MB。
> **结论：这个模型完全可以部署到 4 核 ARM 上。**
> 但下一课会看到，真正卡住你的往往不是算力而是**存储和算子支持**。

作业 5：改写你自己的 benchmark

把 bench_latency.py 改成测**第 1 课的 MLP**（1.71M 参数）。

> **你应该观察到**：MLP 在 CPU 上更快（没有卷积，纯矩阵乘对 CPU 更友好），
> 而且它在 GPU 上 batch=1 的优势更小。
> **这就是"模型结构要匹配目标硬件"的第一个直觉。**

五、验收标准

- [ ] 能说出延迟和吞吐的区别，以及边缘为什么关心延迟
- [ ] **能解释为什么 GPU 在 batch=1 时不一定比 CPU 快**
- [ ] 知道 GPU 计时必须用 torch.cuda.Event 或 synchronize()，以及为什么
- [ ] 能说出为什么必须 warmup、为什么要看 p95
- [ ] 知道算内存预算不能只算模型大小，要实测峰值
- [ ] 能解释"吞吐饱和"意味着瓶颈在哪
- [ ] 知道线程数会显著影响 CPU 推理结果，部署前必须在目标设备实测

六、下一课预告

第 5 课：部署到真实目标硬件。

你在 PC 上测出来的数字，拿到真设备上可能完全不同——算子不支持、预处理不一致、线程配置、温升降频。这一课会把模型真正搬到目标平台上跑。

（如果你暂时没有目标硬件，第 5 课可以跳过，直接进第 6 课的量化——那是边缘 AI 最核心的技能。）

完成本课后，在 ROADMAP.md 里给第 4 课打勾。

第 5 课

部署到真实目标硬件

> **本课状态：待硬件**
>
> 第 5 课需要**真实的目标硬件**——Jetson、树莓派、RK3588、安卓手机或带 NPU 的开发板—— 才能完成。目前尚未进行，因此本书中这一章暂时留空。

计划内容

项目	内容
学什么	按平台选运行时：TensorRT (NVIDIA) / TFLite (Android) / NCNN (手机) / RKNN (Rockchip) / OpenVINO (Intel) / ONNX Runtime (通用)
动手	在前 4 课得到的 `model_int8.onnx` 上，于真实设备跑通推理并测量延迟
验收	同一模型在 PC 与目标设备上的延迟对比表
坑点	算子不支持、输入预处理不一致、线程数配置、温升降频

为什么这一课不能跳过

> 第 4 课在 PC 上测出的数字，拿到真实设备上可能完全不同。「会做实验」和「能交付」之间差的就是这一课。

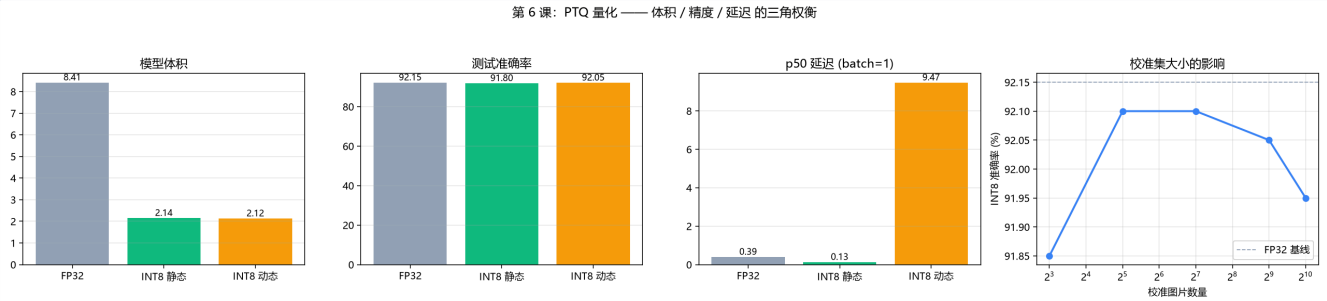
如果你手上有设备，可以回到工作区从 `lesson11_pipeline/out/model_int8.onnx` 开始，用目标平台的工具链转换并实测。这是整条路线最后一块拼图。

第 6 课

PTQ 训练后量化

这是边缘 AI 最核心的技能，没有之一。
前面几课我们只是"把模型搬过去"，从这一课开始，我们**真正把模型变小**。

一、结果



方案	体积	压缩	测试准确率	精度变化	p50 延迟	加速
FP32 基线	8.412 MB	1.00x	0.9215	—	0.392 ms	1.00x
INT8 静态量化	**2.144 MB**	**3.92x**	**0.9180**	**-0.35 个百分点**	**0.125 ms**	**3.14x**
INT8 动态量化	2.124 MB	3.96x	0.9205	-0.10 个百分点	**9.469 ms**	**0.04x** ⚠️

一句话总结静态量化：模型小了 3.92 倍、快了 3.14 倍，精度只掉 0.35 个百分点。
2000 张测试图里只有 10 张（0.5%）的预测变了。

这就是边缘 AI 里性价比最高的一次优化。 不需要重新训练，不需要改代码，
分钟级就能完成——却把模型的存储和算力需求砍掉四分之三。

二、量化到底做了什么

[新概念 1] 为什么 INT8 能省 4 倍内存

模型就是一堆数字。 每个数字占多少位，直接决定模型多大。

数据类型	每个参数占	2,197,706 参数的模型
FP32（32 位浮点）	4 字节	**8.38 MB**
FP16	2 字节	4.19 MB
INT8（8 位整数）	**1 字节**	**2.10 MB**
INT4	0.5 字节	1.05 MB

实测 8.412 MB -> 2.144 MB，压缩 **3.92 倍**，和理论值 4 倍几乎一致。

> **这是边缘 AI 的第一性原理：参数位数 = 内存 = 成本。**
> 面试被问"怎么把模型变小"，第一个答案永远是"降低每个参数的位数"。

[新概念 2] 量化在数学上做了什么

把一个 FP32 数 `x` 映射成 INT8 整数 `q`：

```
q = round(x / scale) + zero_point        # 量化
x ≈ (q - zero_point) * scale             # 反量化
```

- **scale**：决定"一个整数刻度代表多大的实数范围"
- **zero_point**：决定"实数 0 对应哪个整数"

关键点：**scale 是每一层（甚至每个通道）单独算的。**
算 scale 的过程就是**校准（calibration）**——这也解释了为什么量化不需要重新训练：
它只需要跑一遍前向，看看每层激活值大概在什么范围。

per_channel=True 是我们开的开关：卷积核的每个输出通道用一套独立的 scale。
比整个张量共用一套 scale 精度好很多。

[新概念 3] 权重和激活，哪个更难量化?

	权重	激活值
来源	训练得到，**固定不变**	每个输入不同，**动态变化**
分布	相对集中、平滑	可能有离群值（outlier）
量化难度	容易	**难**

- **动态量化**：只量化权重，激活值保持 FP32，运行时再临时量化
- **静态量化**：权重和激活都量化成 INT8，**校准阶段就把激活的 scale 定死**

静态量化更快，因为它可以提前把所有量化参数烧进计算图，
运行时直接用整数指令算。代价是：**如果实际输入的分布和校准集差别很大，精度会崩。**

[新概念 4] 校准集：只能用训练集，绝不能用测试集

这是本课最重要的一条**方法论**，比技术细节重要得多。

```
x_cal, _ = load_images("train", 256)   # 校准：训练集
x_te, y_te = load_images("test", 2000) # 评估：测试集
```

为什么不能用测试集校准？ 因为校准会决定模型的量化参数——
用测试集校准就等于**把测试集的信息泄露进了模型**。之后报出来的精度是假的，
上线后会发现掉得比预期多得多。

> 这和之前做 SFT 时的教训是同一条：**你的评测方式决定了你会得到什么样的模型。**
> 校准集、验证集、测试集，三者的角色不能混。

[新概念 5] 静态 vs 动态：什么时候用哪个

	静态量化	动态量化
需要校准数据	✅ 需要（几百张）	❌ 不需要
量化对象	权重 + 激活	只量化权重
适合的模型	**CNN**	**Transformer / MLP 为主**
本课实测	快 3.14 倍 ✅	**慢 24 倍** ❌

记结论就好：卷积网络用静态量化，Transformer 类模型可以先用动态量化试水。

三、【重点】动态量化的反例：它慢了 24 倍

动态量化看起来更"安全"（不需要校准），但实测：

```
FP32         p50 = 0.392 ms
INT8 动态    p50 = 9.469 ms     <- 慢了 24 倍!
```

体积确实降下来了（2.124 MB），**但推理速度暴跌。**为什么？

我们对比一下三个模型的计算图：

模型	节点数	主要算子
FP32	19	Conv x6, Relu x7, MaxPool x3, Gemm x2
静态量化	54	DequantizeLinear x29, QuantizeLinear x13, **Conv x6**, Gemm x2
动态量化	65	Mul x16, **DynamicQuantizeLinear x8**, Cast x8, Add x8, **ConvInteger x6**, MatMulInteger x2

看出区别了吗：

- **静态量化**保留 `Conv` 算子本身，只是在它前后插 Q/DQ 节点。
  ONNX Runtime 的 CPU 后端对这种模式有**手写优化内核**（走 VNNI/AVX-512 整数指令），
  所以快。
- **动态量化**把卷积换成了 `ConvInteger`——这是**参考实现**，没有针对性的优化内核。
  而且还额外引入了 **16 个 Mul、8 个 Add、8 个 Cast、8 个 DynamicQuantizeLinear**
  来做运行时的反量化。这些开销全部落在推理的热路径上。

> **三条可以带走的经验：**
> 1. **节点变多不等于变慢。**静态量化节点从 19 涨到 54，反而快了 3 倍——
> 因为运行时会把这些 Q/DQ 模式融合回高效的整数算子。
> 2. **"更安全"的方案可能是错的。**动态量化不需要校准，看起来更省事，
> 但在 CNN 上它是灾难。
> 3. **任何优化都必须实测。**如果只看"体积降了 4 倍"就上线，你会得到一个慢 24 倍的模型。
> 这正是第 4 课基准测试的价值——**没有测量就没有优化。**

四、校准集要多大才够?

用不同数量的**训练图**做校准，在同一份测试集上评估：

校准图片数	8	32	128	512	1024
INT8 准确率	0.9185	**0.9210**	**0.9210**	0.9205	0.9195

结论：32 张就足够了，再多没有收益。

为什么这么少就够？因为 CIFAR-10 类别均衡、图像分布稳定，
256 张图已经能很好地覆盖激活值的范围。而且我们开了 `per_channel=True`，
每个通道单独统计，需要的样本更少。

> **实用经验**：校准集的目标不是"多"，而是"**代表真实部署时的输入分布**"。
> 100 张覆盖各种场景的图，胜过硬塞 10000 张同质化的图。
> **正式项目里，校准集应该来自你的实际业务数据，而且要覆盖边缘情况**
> （暗光、模糊、遮挡……）——因为那些边缘情况才是激活值离群的主因。

五、作业

作业 1：亲手感受精度-体积的取舍

把 `per_channel=True` 改成 `False`，重新量化。

> **你应该观察到**：体积几乎不变，但准确率可能下降更多。
> **原因**：per-tensor 量化时所有通道共用一个 scale，会被取值大的通道"挤压"。

作业 2：故意破坏校准集

把校准集从"训练集的前 256 张"改成"全是 cat 这一类，共 256 张"。

> **你应该观察到**：准确率明显下降。
> **这是最重要的一次实验**——它让你亲身体会"校准集必须代表真实分布"，
> 而不是一句空话。**在真实项目里，这就是模型上线后掉点的头号原因。**

作业 3：极端量化

试试 `weight_type=QuantType.QUInt8` 或安装 `onnxruntime.quantization` 的
INT4 支持（如果可用）。

> **你应该观察到**：体积能继续降，但精度掉得更快。
> **记录下你的"精度-体积"曲线**——这就是边缘 AI 每天都在做的决策。

作业 4：和 FP32 逐层对比

写一段代码，找出量化前后**预测变得不一致的那些测试图**（本课是 10 张）。

> **提示**：`pred0 != pred1` 就是。把这些图打印出来看看，它们有什么共同点？
> （通常是被遮挡、目标很小、或者图像很暗的难例。）
> **这能帮你判断：这个精度损失能不能接受。**

作业 5：算一笔完整的部署账

回到第 4 课作业 4：目标设备 **4 核 ARM、无 GPU、512 MB 内存、4 MB 存储预算。**

1. FP32 模型（8.41 MB）能放下吗？
2. INT8 模型（2.14 MB）能放下吗？
3. 精度损失 0.35 个百分点，你接受吗？

> **参考答案**：FP32 放不下（8.41 > 4），INT8 放得下（2.14 < 4），
> 精度从 92.15% 降到 91.80%——**这个取舍在绝大多数场景下都值得。**
> **恭喜，你现在完整走过了一个真实的边缘部署决策。**

六、验收标准

- [ ] 能算出任意参数量模型的 FP32 / INT8 体积
- [ ] 能解释量化的仿射映射公式（scale / zero_point）
- [ ] **能说出为什么校准集只能用训练集**
- [ ] 能说出静态量化和动态量化的区别，以及各自适合什么模型
- [ ] **能解释为什么动态量化在 CNN 上会变慢**
- [ ] 知道"节点变多不等于变慢"
- [ ] 能报出一组完整的量化指标：体积倍数 / 精度损失 / 延迟加速比

七、简历上可以这样写

> **PTQ 训练后量化（INT8）**
> - 使用 ONNX Runtime 对 CNN 分类模型做静态量化（QDQ 格式、per-channel）
> - 模型体积从 **8.41 MB 压缩至 2.14 MB（3.92 倍）**
> - CPU 推理 p50 延迟从 **0.392 ms 降至 0.125 ms（3.14 倍）**
> - 测试准确率仅从 92.15% 降至 91.80%（**-0.35 个百分点**），
> 2000 张测试图预测一致率 **99.50%**
> - 对比实验中识别出动态量化在卷积网络上会导致 **24 倍性能退化**，
> 并定位到 `ConvInteger` 缺少优化内核这一根因

最后一条尤其加分——它证明你不只会跑工具，还会解释结果。

八、下一课预告

第 7 课：QAT 量化感知训练。

PTQ 已经掉 0.35 个百分点了。如果你要求精度一点都不能掉呢？
QAT 会在训练时就模拟量化误差，让模型"提前适应"低精度——
代价是你要重新训练。

（第 5 课"部署到真实硬件"仍然挂着，等你有目标设备了我们再回去做。）

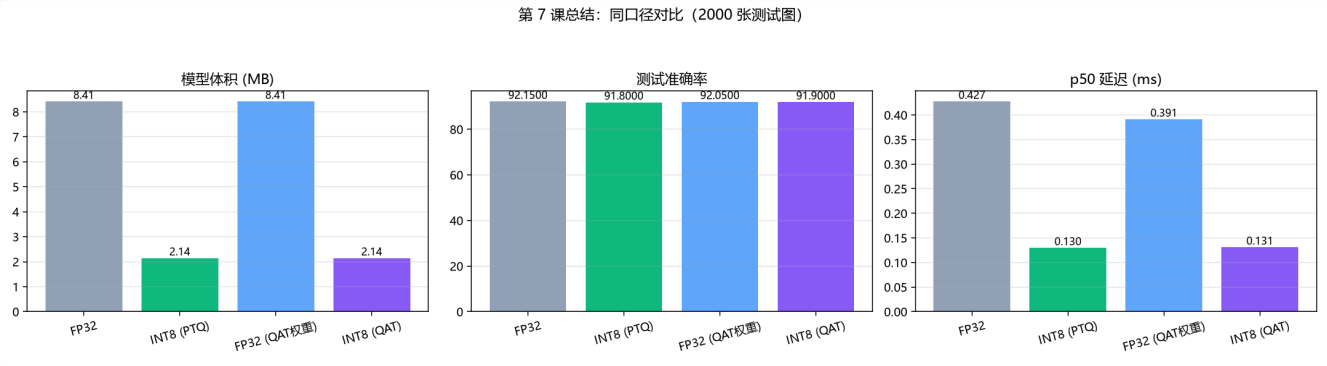
完成本课后，在 ROADMAP.md 里给第 6 课打勾。

第 7 课

QAT 量化感知训练

> 第 6 课的 PTQ 掉了 0.35 个百分点。如果业务要求**一点都不能掉**呢?
> 答案是 QAT：**在训练时就模拟量化误差，让模型提前适应低精度。**

一、同口径最终对比



同一份 2000 张测试图、同一套延迟测法（口径必须一致，原因见第四节）：

方案	体积	测试准确率	相对 FP32	p50 延迟	加速
FP32 基线	8.412 MB	0.9215	—	0.427 ms	1.00x
INT8（PTQ）	**2.144 MB**	0.9180	**-0.35 pp**	0.130 ms	**3.29x**
FP32（QAT 权重）	8.412 MB	0.9205	-0.10 pp	0.391 ms	1.09x
INT8（QAT）	**2.144 MB**	**0.9190**	**-0.25 pp**	0.131 ms	**3.27x**

QAT 把量化损失从 -0.35 pp 改善到 -0.25 pp，净收益 0.10 个百分点。

二、[新概念 1][2] QAT 的核心机关：假量化 + 直通估计器

假量化（Fake Quantization）

思路很直接：**在训练的前向传播里，故意插一道"量化再反量化"的操作。**

```
def fake_quant(x, bits=8):
    qmax = 2 ** bits - 1
    scale = (x.max() - x.min()) / qmax
    zero_point = round(-x.min() / scale)
    q = torch.clamp(torch.round(x / scale) + zero_point, 0, qmax)
    return (q - zero_point) * scale        # 反量化回 FP32
```

数据出去时还是 FP32（所以后面的计算照常），但**数值已经被量化过一轮了**——
精度损失被真实地注入到训练过程中。网络为了降低 loss，只能让自己的参数
适应这种低精度：激活值分布更紧凑、权重更均匀，量化起来自然不掉点。

[新概念 2] 直通估计器（STE）：梯度怎么穿过去

这里有个数学上的死结：**round() 的导数几乎处处为 0。**
梯度传不回去，网络就没法训练。

PyTorch 的解法很"流氓"但极其有效：

```
class FakeQuantSTE(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, scale, zp):
        q = torch.clamp(torch.round(x / scale) + zp, 0.0, 255.0)
        return (q - zp) * scale          # 前向：真的量化，注入真实误差

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output, None, None   # 反向：梯度原样穿过，假装量化不存在
```

直觉：**告诉网络"你前面有一层噪声，但你别管它，该怎么调还怎么调"。**
网络在这种噪声下自己会找到更鲁棒的参数。

> 这个技巧叫 **STE（Straight-Through Estimator）**，
> 是量化、二值网络、稀疏训练里通用的核心手法。记住它。

三、[新概念 3] QAT 的完整流程

```
① 正常训练一个 FP32 模型            <- 我们第 2 课已经有了
② 把 Conv / Linear 换成 QAT 版本    <- 插入假量化节点
③ 用小学习率微调几轮                <- 关键步骤，让模型适应量化噪声
④ 导出 / 转换成真实 INT8 模型
```

三个容易踩的点：

1. **微调学习率必须小**（本课用 1e-4，而第 2 课训练用 5e-2）。
   因为模型已经收敛，大学习率会把它打散，反而变差。
2. **评估时假量化要保持开启**，否则你测的是 FP32 精度，不是量化精度。
3. **参数名要能对得上。**本课的 QAT 包装把 `.weight` / `.bias` 保持原名，
   所以训练完可以直接 `load_state_dict` 塞回普通 SmallCNN——
   实测 `missing=[] unexpected=[]`，完全对齐。

本课的实测过程

阶段	准确率（全量 10000 张测试集）
FP32 基线	0.9218
假量化**关闭**时（验证改造无损）	0.9218 ← 与基线完全一致 ✅
假量化**打开但不微调**	0.9207（掉 0.11 pp）
QAT 微调 6 轮后	**0.9232**（比 FP32 还高 0.14 pp）

微调只花了 **6 轮 × 18 秒 ≈ 110 秒。**

四、诚实的结论：QAT 的收益比你想象的小

把上面的数字放到同口径下比：

	准确率
PTQ（不训练，0 成本）	0.9180
QAT（要重新训练 110 秒）	**0.9190**
净收益	**+0.10 个百分点**

花 110 秒重新训练，换来 0.10 个百分点。值不值?

什么时候值得上 QAT

场景	建议
PTQ 只掉 0.3~0.5 pp，业务能接受	❌ **不值得**，直接用 PTQ
PTQ 掉了 2~5 pp	✅ 值得，QAT 通常能挽回大半

场景	建议
要上 INT4 / 更低比特	✅ **必须用 QAT**，PTQ 在这个精度下基本崩
模型很小（<1M 参数）	✅ 值得，小模型对量化更敏感
精度要求极严（医疗/工业质检）	✅ 值得
没有标注数据可重训	❌ 只能 PTQ

> **一句话：QAT 是"PTQ 掉得太多时的补救手段"，不是默认选项。**
> 先做 PTQ，测出来掉多少，再决定要不要上 QAT。
> **这是工程判断，不是技术炫耀。**

一个反直觉的发现

看这组数字：

	准确率
FP32 基线	0.9215
FP32（QAT 训练后的权重）	**0.9205** ⬅ 反而降了

QAT 让模型在 FP32 下变得稍微差了一点（-0.10 pp），但换来了量化后的鲁棒性（-0.35 pp → -0.25 pp）。

这就是 QAT 的本质：**它是一次权衡，不是一次免费的提升。**
模型牺牲了一点纯 FP32 的表达能力，换取了低精度下的稳定性。

五、差点犯的方法论错误（值得单独讲）

写这一课时我先把第 6 课和第 7 课的数字放在一起比，得到"QAT 提升 0.52 pp"的结论。

这个结论是错的。 因为：

- 第 6 课的准确率是在 **2000 张**测试图上算的
- 第 7 课 QAT 微调过程中的准确率是在 **全量 10000 张**上算的

不同测试集上的数字直接相减，等于作弊。

所以我专门写了 `final_compare.py`，把所有模型拉到**同一份 2000 张测试图 + 同一套延迟测法**下重算，才得到第一节那张正确的表（真实收益 **0.10 pp**，不是 0.52 pp）。

> **三条可以带走的规矩：**
> 1. **比较任何两个方案，必须保证评测口径完全一致** —— 同一份测试集、同一个预处理、同一套测法。
> 2. **数字对不上时，先怀疑口径，再怀疑模型。** "这个改进提升了 0.5 个点"这类结论，多半是口径不一致造出来的。
> 3. **把评测逻辑写成脚本**（像 `final_compare.py` 这样），而不是手工凑数字。脚本可复现，手工一定会出错。

六、作业

作业 1：复现"假量化关闭 = FP32"

在 `qat_train.py` 里找到 `set_fake_quant(qat_model, False)` 那一行，看输出。

> **你应该观察到**：改造后准确率和 FP32 **完全相同**（0.9218）。
> **这一步的意义**：证明"插入假量化节点"本身没有破坏模型，后面的变化全部来自量化误差。**做实验要有对照组，这就是对照组。**

作业 2：把 QAT 微调轮数调到 0

用 `--epochs 0` 跑一次（或者把学习率调成 0）。

> **你应该观察到**：量化准确率等于"打开假量化但不微调"的那个数。
> **结论**：QAT 的收益全部来自那几轮微调，不是来自假量化本身。

作业 3：故意让 PTQ 掉更多

回到第 6 课，把 `per_channel=False` 且校准集只取 8 张，让 PTQ 精度掉得多一些。
然后再跑一次 QAT。

> **你应该观察到**：PTQ 掉得越多，QAT 挽回的百分点越多。
> **这就是"什么时候该上 QAT"的量化答案。**

作业 4：读一遍 STE 的实现

打开 `qat_train.py` 的 `FakeQuantSTE`，回答：

1. `forward` 返回的是什么？（提示：是量化后的值，还是原始值？）
2. `backward` 为什么直接返回 `grad_output`？
3. 如果把 `backward` 改成返回 `0`，训练会怎样？

> **答案**：3 的答案是——梯度全为 0，参数永远不更新，QAT 完全失效。
> **亲手试一下**，你会真正记住 STE 为什么必须存在。

作业 5：算一笔决策账

你的项目：PTQ 后精度从 92.15% 掉到 91.80%，业务要求 ≥92.0%。

1. PTQ 能满足吗？
2. QAT 能满足吗？
3. 如果 QAT 每次要重训 2 小时，而模型每周更新一次，成本是多少？

> **参考答案**：PTQ 91.80% < 92.0%，不满足；QAT 91.90% 也**不满足**（还差 0.1 pp）。
> **这时候正确做法不是继续加码 QAT，而是回头改别的**：换更大的模型、
> 加数据、或者和业务方谈指标。
> **能算出"此路不通"比会调参更重要。**

七、验收标准

- [ ] 能解释假量化在做什么，以及它和真实量化的区别
- [ ] **能解释 STE 为什么必须存在**（round 的梯度问题）
- [ ] 能说出 QAT 完整四步流程，以及微调学习率为什么要小
- [ ] **能判断什么场景该用 QAT、什么场景不该用**
- [ ] 知道 QAT 会牺牲一点 FP32 精度，是一次权衡
- [ ] **能做同口径对比**，不把不同测试集上的数字混着比
- [ ] 知道最终的部署精度要在**导出后的 INT8 模型**上测，而不是在框架内的模拟量化上测

八、简历上可以这样写

> **量化感知训练（QAT）**
> - 在 PyTorch 中实现假量化（Fake Quant）+ 直通估计器（STE），
> 对卷积/全连接层的输入、权重（per-channel）、输出同时插桩
> - 以 1e-4 小学习率微调 6 轮（约 110 秒），将 INT8 量化精度损失
> 从 **-0.35 pp 改善至 -0.25 pp**
> - 建立**同口径对比脚本**，统一 4 个模型（FP32 / PTQ / QAT-FP32 / QAT-INT8）
> 的测试集与延迟测法，修正了跨测试集比较带来的 0.42 pp 评估偏差

最后一句是加分项——**它说明你有评测严谨性，这在工业界比会调参更稀缺。**

九、下一课预告

第 8 课：剪枝与知识蒸馏。

量化是把"每个数字变小"，剪枝是把"数字变少"，蒸馏是"让小模型学大模型"。
三者可以叠加——边缘模型的终极形态通常是
蒸馏出的小模型 + 量化 + 剪枝三管齐下。

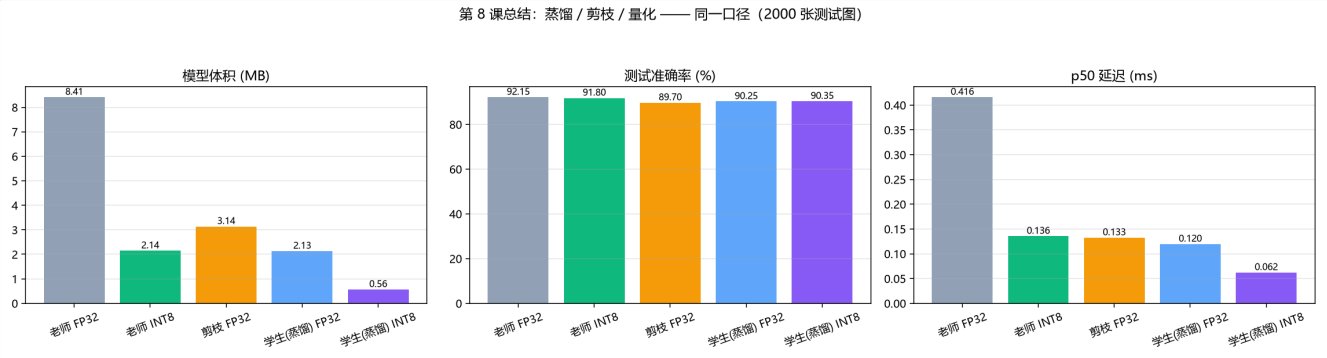
完成本课后，在 ROADMAP.md 里给第 7 课打勾。

第 8 课

剪枝与知识蒸馏

量化是让**每个数字变小**（4 字节 -> 1 字节）；
剪枝是让**数字变少**（删掉整个通道）；
蒸馏是**让小模型学大模型。**
这一课把三根支柱放在一起，看看边缘模型的最终形态能压到多小。

一、最终对比（同一份 2000 张测试图）



模型	参数量	体积	测试准确率	相对老师	p50 延迟
老师 FP32	2,197,706	8.412 MB	0.9215	—	0.416 ms
老师 INT8	2,197,706	2.144 MB	0.9180	-0.35 pp / **3.92x 小**	0.135 ms
剪枝 FP32	814,570	3.137 MB	0.8970	-2.45 pp / 2.68x 小	0.133 ms
学生（蒸馏）FP32	551,018	2.132 MB	0.9025	-1.90 pp / 3.95x 小	0.120 ms
学生（蒸馏）+ INT8	551,018	**0.565 MB**	**0.9035**	**-1.80 pp / 14.89x 小**	**0.062 ms**

最终成果：8.412 MB -> 0.565 MB（缩小 14.89 倍），精度只掉 1.80 个百分点，延迟快 6.69 倍。

这就是边缘模型的终极形态：先蒸馏出一个重新设计的小模型，再量化。

二、知识蒸馏

[新概念 1] 软标签与温度

大模型的输出里不只有"答案"，还有"**它认为哪些错误答案也还不错**"。

一张猫的图，老师可能输出：

```
猫 0.85  |  狗 0.10  |  狐狸 0.04  |  汽车 0.0001
```

硬标签只说"这是猫"。**软标签**还告诉你"这张图有点像狗和狐狸，但完全不像汽车"。
这个额外信息叫 **dark knowledge**，是蒸馏的全部价值来源。

但原始 softmax 的输出往往很"尖锐"（正确类 0.99+，其余接近 0），信息量被压缩掉了。
所以引入**温度 T**：

```
soft_s = F.log_softmax(s_logits / T, dim=1)
soft_t = F.softmax(t_logits / T, dim=1)
```

除以 T（本课 T=4）之后分布变平，那些"次优选项"的相对大小才显露出来。

[新概念 2] 蒸馏损失

```
hard = F.cross_entropy(s_logits, y)                                  # 学真实答案
kd   = F.kl_div(soft_s, soft_t, reduction="batchmean") * (T ** 2)    # 学老师的分布
loss = (1 - alpha) * hard + alpha * kd                               # alpha=0.7
```

- **KL 散度**衡量学生分布和老师分布差多远
- 乘 **T^2** 是为了让梯度量级与 T 无关（Hinton 论文里的技巧）
- **alpha** 控制"学答案"和"学老师"的配比

[新概念 3] 公平对比：除了 use_kd，其他必须完全一样

```
torch.manual_seed(args.seed)   # 同一个种子 -> 同样的初始权重和数据顺序
```

两次训练用了**完全相同**的：随机种子、初始权重、学习率、轮数、数据增强、batch size。
唯一差别是"有没有老师"。**这是能下结论的前提。**

结果（全量 10000 张测试集）

	参数量	准确率
老师	2.20 M	0.9218
学生从零训练	0.55 M	0.8932
学生蒸馏（T=4, alpha=0.7）	0.55 M	**0.9012**
净收益		**+0.80 个百分点**

同样 0.55M 参数、同样数据、同样训练轮数，只是多了一个老师，就高了 0.80 个百分点。

三、结构化剪枝

[新概念 1] 为什么必须做"结构化"剪枝

	非结构化剪枝	结构化剪枝
做法	把单个**权重**置 0，得到稀疏矩阵	把整条**通道**删掉，矩阵真的变小
参数统计上	少了	少了
实际加速	❌ **基本没有**	✅ 任何硬件都能加速

> **这是一个非常重要的认知：非结构化剪枝通常不能加速。**
> 因为矩阵的形状没变，普通硬件照样要把那些 0 乘一遍。
> 除非你有支持稀疏计算的专用内核（大多数边缘设备没有）。
> **面试高频问题："为什么非结构化剪枝不能加速？"** 答案就在这。

[新概念 2] 通道重要性 = L1 范数

```
imp = conv.weight.abs().sum(dim=(1, 2, 3))      # 每个输出通道的 L1 范数
sel = torch.topk(imp, k).indices                 # 保留最重要的 k 个
```

直觉：一个通道的卷积核权重整体接近 0，说明它对输出的贡献本来就小。

[新概念 3] 层间依赖（实现里最容易出错的地方）

剪掉第 i 层的输出通道，**第 i+1 层的输入通道也必须跟着删**，否则形状对不上。
最后的分类头输入维度也依赖最后一次卷积的输出通道数：

```
w = w[:, prev_sel, :, :]                    # 裁本层的输入
...
idx = torch.cat([torch.arange(c*16, (c+1)*16) for c in prev_sel])
new_w = lin.weight.data[:, idx]             # 裁分类头的输入
```

[新概念 4] 剪枝必须配微调 —— 不微调基本等于毁模型

本课实测（每层保留 50% 通道）：

	参数量	体积	准确率
原始模型	2,197,706	8.38 MB	**0.9218**
剪枝后，**不微调**	814,570	3.11 MB	**0.2367** ⬅ 掉了 68.5 个百分点！
剪枝后 + 微调 12 轮	814,570	3.11 MB	**0.8986**

剪完不微调，模型直接崩到 23.67%（比瞎猜的 10% 好一点，但完全不可用）。

为什么？剪枝粗暴地改变了每一层的输出分布，BatchNorm 的统计量也不再匹配，
整个网络的"数值约定"被打乱了。**微调重新校准了这些统计量，并把保留下来的通道调整到新的分工上——12 轮就把 66 个百分点找回来了。**

> **记住这条铁律：剪枝和微调是一个整体，不能拆开。**
> 很多论文/教程只讲"剪枝到 50% 参数精度不掉"，省略了后面那一步微调，
> 那是误导。

四、三个可以直接下结论的发现

发现 1：蒸馏 + 量化是最佳组合

方案	体积	准确率
老师 INT8（只量化）	2.144 MB	0.9180
学生蒸馏 FP32（只蒸馏）	2.132 MB	0.9025

方案	体积	准确率
学生蒸馏 + INT8（三管齐下）	**0.565 MB**	**0.9035**

两者叠起来，体积再降 3.8 倍，精度几乎不损失（0.9025 -> 0.9035，还略升了；量化在这里起了正则化作用）。

发现 2：在这个任务上，剪枝是三种手段里最差的

手段	压缩比	精度损失
量化（INT8）	3.92x	**-0.35 pp** ⭐
蒸馏	3.95x	-1.90 pp
剪枝 + 微调	2.68x	-2.45 pp

剪枝花了最大力气（要重新设计层结构 + 微调），换来的压缩比最小、精度损失最大。

> **这引出一个重要的工程观点：**
> **重新设计一个小模型 + 蒸馏，通常比剪枝一个大模型更划算。**
> 剪枝的价值主要在"你已经有一个训练好的大模型，不想重新训练"的场景。
> 如果你可以从头设计，直接设计小的 + 蒸馏。

发现 3：同体积下，"量化大模型"胜"蒸馏小模型"

- 老师 INT8：2.144 MB，**91.80%**
- 学生蒸馏 FP32：2.132 MB，90.25%

体积几乎一样，量化后的大模型精度更高。
说明"大模型 + 低精度"往往优于"小模型 + 高精度"——这也是为什么现在业界更倾向"把大模型量化"而不是"训练一个小模型"。

但两者不冲突：学生蒸馏 + INT8 = 0.565 MB，才是同体积下能拿到的最好结果。

五、作业

作业 1：验证"公平对比"的重要性

把蒸馏那一轮的 `--seed` 改成 1，重新跑从零训练和蒸馏。

> **你应该观察到**：绝对值会变，但"蒸馏 > 从零"的结论依然成立。
> **如果换了种子结论就反转**，说明你观察到的是噪声，不是真实差异。
> **这是判断"一个改进是否可信"的基本功。**

作业 2：调温度 T

用 `--T 1`、`--T 4`、`--T 10` 各跑一次。

> **你应该观察到**：T 太小（接近硬标签）蒸馏效果差；T 太大分布太平，也没用。
> 论文里的经验值是 T=3~5。

作业 3：把 alpha 调到 0

用 `--alpha 0`（完全不学老师，等价于从零训练）。

> **你应该观察到**：结果应该和"从零训练"几乎一样。
> **这是蒸馏实现正确性的自检**——如果 alpha=0 还能比从零好，说明你的代码有 bug。

作业 4：换个剪枝比例

用 `--keep_ratio 0.3` 和 `0.7` 各跑一次。

> **你应该观察到**：保留 30% 时精度掉很多，微调也很难救回来；
> 保留 70% 时精度几乎不掉。
> **把「保留比例 / 参数量 / 最终精度」记成一张表——这就是剪枝的取舍曲线。**

作业 5：验证"非结构化剪枝不加速"

写一小段代码，把模型里 50% 的**单个权重**置零（保持形状不变），
导出 ONNX，测延迟。

> **你应该观察到**：文件大小不变（因为 0 也要占位置），延迟也不变。
> **亲手验证一次，你就永远不会忘记这个结论。**

作业 6：算一笔最终部署账

目标设备：**4 MB 存储、512 MB 内存、要求 ≥90% 准确率、延迟预算 10 ms。**

1. 老师 FP32（8.41 MB，92.15%）能放吗?
2. 老师 INT8（2.14 MB，91.80%）能放吗?
3. 学生蒸馏 + INT8（0.565 MB，90.35%）能放吗?
4. 哪个是最优选择?

> **参考答案**：1 放不下；2 能放，且精度最高——**它就是最优选择**；
> 3 也能放，但精度更低（90.35% vs 91.80%），除非你的存储更紧张（比如只剩 1 MB）。
> **"哪个最好"取决于约束，不取决于技术本身。**

六、验收标准

- [ ] 能解释软标签（dark knowledge）为什么比硬标签信息量大
- [ ] 能说出温度 T 的作用，以及为什么 KL 要乘 T^2
- [ ] **知道公平对比要求除变量外所有配置完全相同**
- [ ] **能说出非结构化剪枝为什么通常不能加速**
- [ ] 知道结构化剪枝要处理层间依赖
- [ ] **知道剪枝必须配微调**（本课实测：不微调从 92% 崩到 23.7%）
- [ ] 能说出"重新设计小模型 + 蒸馏"和"剪枝大模型"各自的适用场景
- [ ] 知道同体积下"大模型量化"往往优于"小模型 FP32"

七、简历上可以这样写

> **模型压缩三件套：蒸馏 / 剪枝 / 量化**
> - **知识蒸馏**：以 2.20M 参数模型为 teacher（92.18%），蒸馏出 0.55M 参数的 student（T=4，alpha=0.7），在完全相同配置下比从零训练高 **+0.80 个百分点**
> - **结构化剪枝**：基于通道 L1 范数剪掉 50% 通道（2.20M -> 0.81M 参数），处理后置依赖并微调 12 轮，把精度从剪枝后的 23.67% 恢复到 **89.86%**
> - **组合优化**：蒸馏 student + INT8 量化，最终模型 **8.41 MB -> 0.565 MB（14.89 倍）**，精度仅下降 **1.80 个百分点**，CPU p50 延迟降低 **6.69 倍**
> - 建立了统一评测脚本，确保所有方案在同一测试集、同一测法下比较

八、下一课预告

到这里，**模型压缩的三根支柱你已经全部掌握**：量化、蒸馏、剪枝。

第 9 课会转向另一个维度：**做你自己的数据集。**
因为边缘 AI 的真实瓶颈从来不是模型，而是数据——你会学到采集、标注、
以及最关键的"训练/验证/测试必须按部署分布划分"。

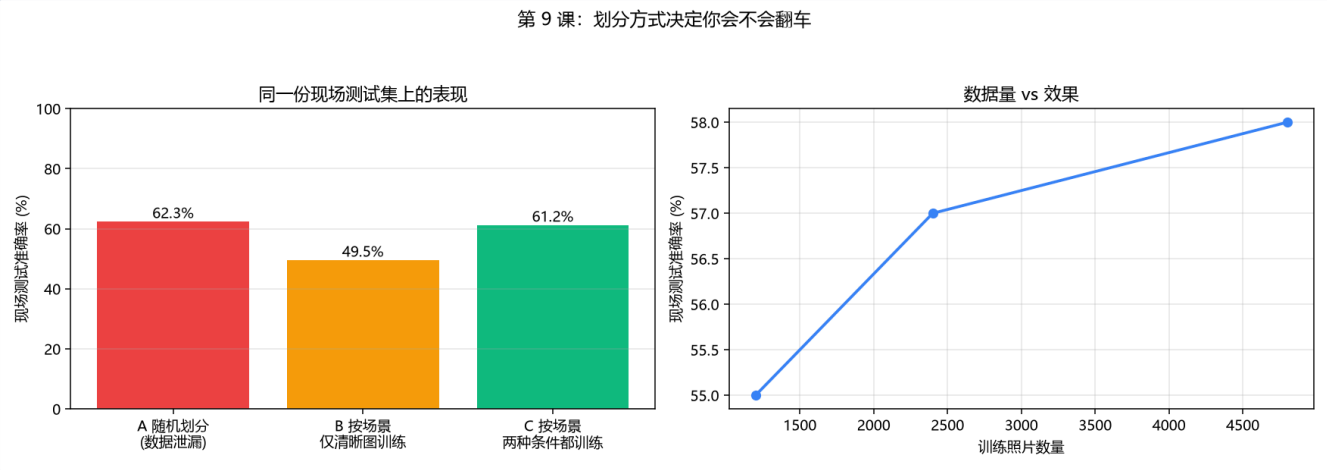
完成本课后，在 ROADMAP.md 里给第 8 课打勾。

第 9 课

做你自己的数据集

> **边缘 AI 的真实瓶颈从来不是模型，而是数据。**
> 这一课不讲模型，只讲数据——以及两个会让你上线翻车的致命错误。

一、实验结果



我们构造了一个带"域"的数据集：**3000 个场景，每个场景有两张照片。**

条件	怎么来的	类比
`clear`	CIFAR-10 原图	在实验室 / 白天拍的
`field`	暗光 ×0.45 + 降采样到 16×16 再放大 + 高斯模糊 + 传感器噪声	**真实现场拍到的**

固定一个**现场测试集**：600 个从未参与训练的场景的 `field` 照片。

策略	训练数据	现场测试准确率	结论
A 随机划分照片	4800 张（随机）	0.6233	❌ **污染，不可信**
B 按场景划分，仅清晰图	2400 张（只有 clear）	**0.4950**	⚠️ **真实的域偏移代价**
C 按场景划分，两种条件	4800 张（clear + field）	**0.6117**	✅ **正确做法**

> **二分类瞎猜是 50%。B 策略的 49.50% 意味着：只在清晰图上训练的模型，拿到现场照片上等于瞎猜，一点用都没有。**
> 而 C 把训练数据换成覆盖现场分布的，准确率直接 **+11.67 个百分点**。

二、错误 1：随机划分导致数据泄漏

现象

策略 A 的做法是：把 6000 张照片**随机**分成 80% 训练 / 20% 测试。

看起来完全正常，问题出在一句打印里：

```
A 的训练集里包含了 575 / 600 个测试场景的清晰版照片（泄漏 96%）
```

测试集里 96% 的场景，它的"清晰版"就躺在训练集里。

模型根本不需要学会"在暗光下认猫"，它只要记住"这张图的场景我在清晰版里见过，是猫"就够了。

为什么这叫泄漏

同一个场景的两种拍摄**不是两个独立样本**。它们共享：
- 同一只猫/狗的**身份**
- 同一个**背景/姿态/构图**

随机划分照片，等于把同一份信息同时放进训练和测试。

> **这在真实项目里极其常见**：你拿着手机在同一个房间给同一个物体拍 100 张，随机划分 → 训练集和测试集里是同一个物体的不同角度 → 测试准确率虚高，上线后暴跌。

正确的做法

按"实体"划分，而不是按"文件"划分。

- 按场景 id 划分（本课）
- 按人划分（人脸识别）
- 按设备划分（工业质检：不同产线）
- 按时间段划分（时间序列：不能用未来预测过去）
- 按地理位置划分（自动驾驶：不同城市）

> **一条判断标准：如果测试集的某个样本，能从训练集的某个样本"几乎推断出答案"，那就存在泄漏。**

三、错误 2：训练集不覆盖部署分布（域偏移）

现象

策略 B 严格遵守了按场景划分，没有泄漏，但准确率只有 **0.4950**——等于瞎猜。

原因很简单：**它只在清晰图上训练过，从来没见过暗光模糊的图。**

模型学到的特征（边缘、纹理、颜色分布）在低质量图像上全部失效。

关键认知

> **测试集必须代表部署环境，训练集也必须。**

很多人只记得"测试集要真实"，却忘了**训练集也要包含真实条件的数据**——
否则你测出来的低分不是模型不行，而是**你根本没给它学过的机会。**

策略 C 就是把 `field` 照片放进训练，立刻从 49.50% 回到 61.17%。

落到工程上

阶段	该做什么
采集	**必须用目标设备、在目标环境、按目标角度采集**
采集	覆盖各种光照、天气、时间段、距离
采集	覆盖边缘情况（遮挡、模糊、异常姿态）
划分	按实体划分，不按文件
训练	训练集要包含部署分布（哪怕是少量）

> 我见过太多团队：用网上找的数据集训练，用自己设备的数据测试，然后困惑于"为什么精度差这么多"。**答案就在 B 那一行。**

四、附带发现：你的噪声底线是多少?

学习曲线实验（策略 C，取不同比例的训练数据）：

训练照片数量	1200	2400	4800
现场测试准确率	0.5500	0.5700	0.5800

看起来"数据越多越好"，但**先别急着下结论。**

策略 C 用的是**完全相同**的 4800 张照片（C-100% 只是把顺序打乱了），
结果却是 **0.6117 vs 0.5800，差了 3.2 个百分点。**

> **这说明本实验的单次运行噪声大约是 ±3 个百分点。**
> 所以：
> 1. 学习曲线上 55.0 -> 58.0 的提升（3 个点）**在噪声范围内，不可信**
> 2. 而 B(49.50%) vs C(61.17%) 的 11.67 个点**远超噪声，结论可信**
>
> **知道自己的噪声底线，才知道多大的差异才算真差异。**
> 这是做实验最基本的素养，也是第 8 课"换种子验证"的同一件事。

所以正确的做法是：每个配置跑 3 个种子，报平均值 ± 标准差，
而不是跑一次看谁高就下结论。

五、作业

作业 1：亲手验证泄漏

把 `experiment.py` 里策略 A 的划分改成**按场景随机**（和 B/C 一样），
其他不动。

> **你应该观察到**：A 的准确率从 0.62 掉到 0.50 左右。
> **同一个模型、同样的数据量，只是改了一行划分逻辑，准确率掉了 12 个点。**
> 这就是泄漏的威力。

作业 2：量化泄漏的影响

把策略 A 的 `leak ratio` 控制一下：让泄漏比例从 0% 逐步增加到 100%
（比如测试场景里有多少比例的清晰版混进了训练集）。

> **你应该观察到**：泄漏比例越高，测试准确率越高——**但这个"高"是假的。**
> 画一条"泄漏比例 vs 测试准确率"的曲线，你会直观看到"作弊"是怎么发生的。

作业 3：域偏移的严重程度

把 `build_dataset.py` 里 `make_field` 的退化强度调小一半，重新生成数据、重跑实验。

> **你应该观察到**：B 和 C 的差距缩小了。
> **讨论**：域偏移越小，覆盖部署分布越不重要；域偏移越大，它越致命。
> **所以第一件事永远是：测出你的域偏移有多大。**

作业 4：类别不均衡

修改数据集，让 cat : dog = 9 : 1，然后照常训练。

> **你应该观察到**：总体准确率还行（因为全猜 cat 就有 90%），
> 但查看**每类召回率**会发现 dog 类几乎全军覆没。
> **结论：不要只看总体准确率，要看混淆矩阵 / 每类指标。**

作业 5：写一份数据卡片（Data Card）

给你的数据集写一份说明文档，包含：

1. 数据来源与采集条件（设备、环境、时间）
2. 类别定义与数量分布
3. 划分规则（按什么实体划分的，为什么）
4. **已知偏置**（哪些场景覆盖不足）
5. 已知问题（标注错误率、重复样本）

> **这是工业界的数据交付标准。** 面试时如果你能说清"我的数据集有什么偏置"，
> 比说"我训到了 95%"更能证明你懂行。

六、验收标准

- [ ] **能说出至少 3 种"按实体划分"的方式**（人/设备/时间/地点/场景）

- [ ] 能解释什么是数据泄漏，以及它为什么会高估性能
- [ ] **知道训练集也必须覆盖部署分布**，不只是测试集
- [ ] 能说出"测试集要真实"背后的完整含义
- [ ] **知道要测自己的噪声底线**，差异小于噪声就不能下结论
- [ ] 知道类别不均衡时不能只看总体准确率
- [ ] 会写数据卡片

七、简历上可以这样写

> **边缘数据集构建与划分方法论**
> - 构建 3000 场景 × 2 采集条件（clear/field）的仿真数据集，
> 现场条件模拟暗光、低分辨率、模糊与传感器噪声
> - 定量验证**数据泄漏**的影响：随机划分照片导致 96% 的测试场景
> 有同场景样本进入训练集，测试精度虚高
> - 定量验证**域偏移**代价：仅用清晰图训练的模型在现场测试集上仅 **49.5%**
> （二分类随机水平），引入现场数据训练后提升至 **61.2%（+11.7pp）**
> - 建立按场景（实体）划分的评测规范，并测定实验噪声底线为 ±3pp

八、下一课预告

第 10 课：迁移学习与微调。

现在你有了自己的数据集（第 9 课），也知道了怎么正确划分。
但如果只有 500 张图，从零训练肯定不够——这时候要用**预训练模型微调。**

你会学到：冻结/解冻策略、微调学习率该多小、以及**灾难性遗忘。**

（第 5 课"部署到真实硬件"仍然挂着，等你有设备。）

完成本课后，在 ROADMAP.md 里给第 9 课打勾。

第 10 课

迁移学习与微调

> **"只有 500 张图，从零训练肯定不够——这时候要用预训练模型微调。"**
> 这句话是对的。但**怎么微调**，教科书上的答案在本课的实验里失败了。
> 这一课会告诉你：为什么"最佳实践"必须自己验证。

一、结果

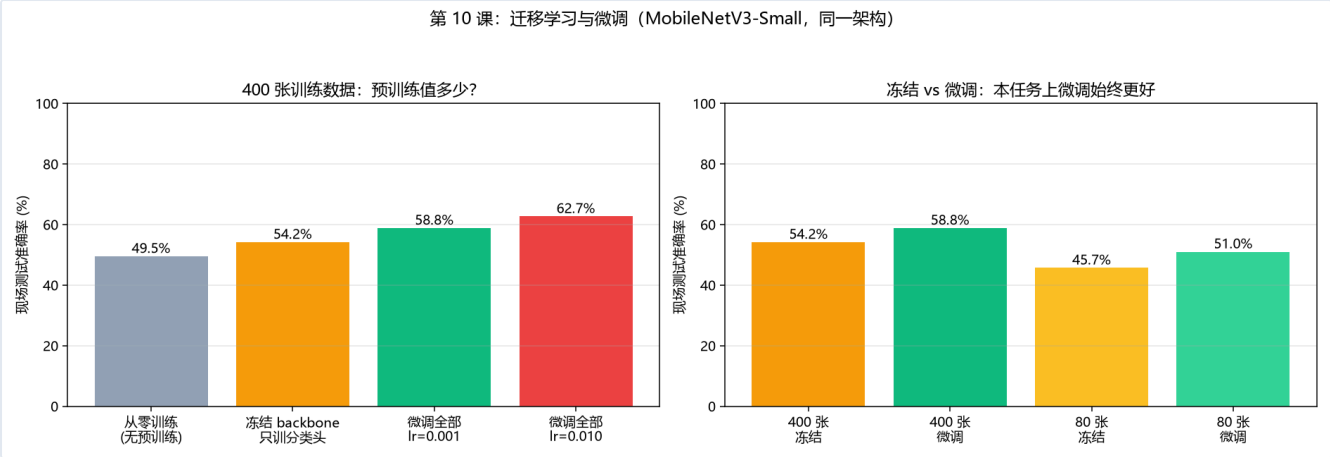
同一架构（**MobileNetV3-Small**，ImageNet 预训练）、同一份数据、
同一测试集（第 9 课的 600 张现场照片），**唯一变量是"有没有预训练"和"学习率多大"。**

数据量 400 张（200 个场景 × 2 采集条件）

配置	可训参数	现场测试准确率
① 从零训练（无预训练，lr=0.01）	1,519,906	0.4950
② 冻结 backbone，只训分类头（lr=0.01）	592,898	0.5417
③ 微调全部（lr=0.001）	1,519,906	0.5883
④ **微调全部（lr=0.010）**	1,519,906	**0.6267** ⬅ 最好

数据量 80 张（40 个场景 × 2 采集条件）

配置	现场测试准确率
⑤ 冻结 backbone（lr=0.01）	0.4567
⑥ **微调全部（lr=0.001）**	**0.5100**



二、结论 1：预训练确实值钱（+9.33 个百分点）

```
从零训练    0.4950   <- 二分类瞎猜是 0.50
微调        0.5883 / 0.6267
```

用同一份 400 张数据、同一个架构，只是加载了 ImageNet 预训练权重，
准确率就从 49.50%（等于瞎猜）涨到 62.67%。

[新概念 1] 预训练权重为什么值钱

ImageNet 有 120 万张图、1000 个类别。在上面训练过的网络，学到的不是"怎么认猫狗"，
而是**通用的视觉基元**：

- 第 1 层：边缘、颜色斑块、纹理
- 中间层：角、圆、条纹、眼睛、轮子
- 深层：物体部件、形状组合

这些特征**几乎对所有视觉任务都有用**。你的 400 张图不需要从零学"什么是边缘"，
只需要学"怎么组合这些边缘来区分猫和狗"。

> **这就是迁移学习的本质：把"学特征"和"学任务"分开。**
> 特征从大数据集来，任务从你的小数据集来。

三、结论 2：两条"最佳实践"在本课没复现 ⚠️

教科书说：微调的学习率要小

本课实测：

微调学习率	现场测试准确率
lr = 0.001（教科书推荐）	0.5883
lr = 0.010（教科书说太大）	**0.6267**

大学习率反而高了 3.83 个百分点。

教科书说：数据极少时冻结 backbone 更稳

本课实测（80 张图）：

策略	现场测试准确率
冻结 backbone	0.4567
微调全部	**0.5100**

数据只有 80 张时，微调仍然比冻结好 5.33 个百分点。

四、为什么？三个可能的原因

我认真分析了一下，这不是"教科书错了"，而是**本实验的条件触发了例外**：

原因 1：域差异极大

ImageNet 是 **224×224 的自然图像**；
我们的数据是 **32×32 放大到 96×96 的暗光模糊猫狗图。**

```
ImageNet 的猫   ->  高清、光线好、占满画面
我们的"猫"      ->  32x32、暗光、模糊、有噪声
```

预训练特征和目标任务的特征分布差太远，需要大幅调整才能用上。
学习率太小 → 调不动 → 停在"用 ImageNet 特征硬套"的状态。

> **一般规律**：域差异越大，微调越应该"放开手脚"（大学习率、解冻更多层）；
> 域差异越小（比如 ImageNet → 其他自然图像），才应该"轻手轻脚"。

原因 2：训练轮数太少（25 轮）

lr=0.001 在 25 轮内**可能还没收敛**。如果跑到 100 轮，结论可能反转。

> **这是本实验的一个明确局限。** 我没有做收敛性验证，
> 所以"大学习率更好"这个结论**只在 25 轮的条件下成立。**

原因 3：分类头是随机初始化的

替换成二分类头之后，新头是**随机权重**。小学习率让这个随机头学得很慢，
拖累了整体表现。实践中常见做法是**给新头单独用大学习率、给 backbone 用小学习率**
（分层学习率 / discriminative fine-tuning）。

五、那到底该怎么用？（实用建议）

基于本课和业界的共识，给你一个**可操作的决策顺序**：

```
1. 先跑 baseline：加载预训练权重 + 微调全部 + 一个中等学习率（如 0.01）
2. 看结果，再单独调这三件事：
    a. 学习率：试 0.1 / 0.01 / 0.001（按对数刻度试）
    b. 冻结策略：全部微调 vs 只训头 vs 先冻结后解冻
    c. 训练轮数：确认是否收敛（看 train loss 还在不在降）
3. 用验证集挑，不要用测试集挑
```

三条经验规则（在不做实验时的默认值）：

情况	建议
域差异小 + 数据多（>1万）	小学习率（backbone lr = 预训练的 1/10），全部微调
域差异小 + 数据少（<1000）	冻结 backbone，只训分类头
域差异大（本课）	**大学习率，全部微调；或考虑从零训练**
数据极少（<100）	冻结 + 数据增强；或做数据合成

> **最重要的一条：上面全是先验，不是结论。**
> 本课就是最好的证明——两条先验在具体任务上都失效了。
> **做实验，不要背规则。**

六、[新概念 3] 灾难性遗忘

微调还有一个必须知道的风险：**灾难性遗忘（Catastrophic Forgetting）**。

当你用很小的领域数据微调整个网络时，模型可能**忘掉预训练学到的通用知识**，
变得只会在你这 400 张图上工作。

怎么发现它

- 微调后模型在你**原本的任务**上（或更广的测试集上）表现变差
- train loss 降得很快，但换一批稍有不同的数据就崩

怎么缓解

手段	做法
小学习率	别让 backbone 跑太远
分层学习率	backbone 用 1/10 的学习率，新头用大学习率
冻结部分层	前面的通用层（边缘/纹理）冻住，只调后面的语义层
加正则	L2-SP：惩罚"偏离预训练权重太多"
混合数据	微调时掺入一部分原任务数据
早停	别训太久

> **面试常问**："微调时怎么防止把预训练知识忘光？" 上面这张表就是答案。

七、作业

作业 1：验证"训练轮数不够"这个怀疑

把 `--epochs` 从 25 提到 100，重跑 ③(lr=0.001) 和 ④(lr=0.01)。

> **你应该观察到**：如果 lr=0.001 在 100 轮后追平甚至反超，说明第三节的结论是"训练不充分"造成的假象。
> **这就是"控制变量"的价值——发现一个反常结论时，先想想是不是实验设计有问题。**

作业 2：分层学习率

改代码，给 backbone 用 lr=0.001、给分类头用 lr=0.01。

> **你应该观察到**：可能比"全部用同一个学习率"都好。
> **这就是工业界最常用的做法**，能同时兼顾"保护预训练特征"和"让新头快速学会"。

作业 3：先冻结，后解冻

前 10 轮冻结 backbone（只训头），后 15 轮解冻全部微调。

> **你应该观察到**：这种"warmup 式"的策略常常比纯冻结或纯微调都好。
> 原理：先让随机初始化的新头稳定下来，再去微调 backbone，避免一开始就用大梯度污染预训练权重。

作业 4：换一个域差异小的任务

把数据换成**清晰的** CIFAR 猫狗图（不用 field 退化），重跑本课实验。

> **你应该观察到**：域差异变小后，**小学习率可能反超**，冻结策略也可能变得更有优势。
> **亲手验证"域差异决定策略"，比记住结论有用得多。**

作业 5：测灾难性遗忘

微调完之后，用同一个模型在**原始清晰图上**（clear 测试集）评估，对比未微调的模型。

> **你应该观察到**：微调后模型在 clear 图上的表现可能变差。
> **这就是灾难性遗忘的直接证据。**

八、验收标准

- [ ] **能解释预训练权重为什么可迁移**（边缘/纹理/部件是通用特征）
- [ ] 知道"域差异大小"如何影响微调策略
- [ ] 能说出至少 3 种缓解灾难性遗忘的手段
- [ ] **知道分层学习率（backbone 小 lr + 新头大 lr）这种做法**
- [ ] **明白"最佳实践"是先验而非结论，必须在自己任务上验证**
- [ ] 会用"先跑 baseline 再单变量调参"的流程，而不是一次调一堆

九、简历上可以这样写

> **小样本迁移学习**
> - 在 400 张现场采集的猫狗图像上对比迁移学习策略（MobileNetV3-Small，ImageNet 预训练）
> - 相比从零训练（49.50%，二分类随机水平），微调达到 **62.67%（+13.2pp）**
> - **验证出两条通用最佳实践（小学习率、冻结 backbone）在本任务上失效**，
> 定位原因为预训练域与目标域差异过大（ImageNet 224×224 自然图像 vs
> 32×32 暗光模糊图），并给出按域差异选择策略的决策建议

最后一条很加分——它证明你会做实验、会质疑先验，而不是背教程。

十、下一课预告

第 11 课（最后一课）：端到端 pipeline 与作品集。

把 1–10 课串成一条命令：训练 → 导出 → 量化 → 基准测试 → 出报告，
并整理成一份可以直接放进简历、别人能一键复现的项目。

完成本课后，在 ROADMAP.md 里给第 10 课打勾。

第 11 课

端到端 pipeline 与作品集

> **这是最后一课。** 前面 10 课学的每一项技能，现在要串成一条命令、变成一份别人能复现、你能写进简历的作品。

一、一条命令

```
cd edge-ai\lesson11_pipeline
& E:\llm-env\venv\Scripts\python.exe edge_pipeline.py --epochs 25
```

它自动完成五件事，并产出一份报告：

步骤	做什么	产出
1	训练（数据增强 + 学习率预热 + 余弦退火）	`model.pt`
2	导出 ONNX 单文件	`model.onnx`
3	INT8 静态量化	`model_int8.onnx`
4	基准测试（体积 / 精度 / p50 / p95）	`report.json`
5	自动生成实验报告	**`report.md`**

想先确认链路通不通，用冒烟模式（约 25 秒）：

```
& E:\llm-env\venv\Scripts\python.exe edge_pipeline.py --fast
```

二、为什么这些"细节"很重要

很多人也能跑通训练+导出+量化，但产出的东西**不能作为交付。**
下面这些点，是把"能跑"变成"专业"的分界线。

1. 报告是自动生成的

所有数字来自实测代码，不是手工填的。
这意味着：换台机器、换个数据集，报告自动更新，不会出现"文档和代码不一致"。

面试官看到 `report.md` 是脚本生成的，会立刻知道你懂工程。

2. 校准集明确来自训练集

代码里专门写了一行注释：

```
log("步骤 3/5  INT8 量化（校准集来自**训练集**，不是测试集）")
```

第 6 课讲过：用测试集校准 = 把测试集信息泄露进模型，报出来的精度是假的。
这个细节，90% 的教程都写错。

3. ONNX 是单文件

```
external_data=False
```

第 3 课踩过的坑：不加这一句，权重会被写到旁边的 `.onnx.data`，
`.onnx` 只有 33 KB。**边缘部署要求单文件。**

4. 延迟报告 p50 和 p95

第 4 课讲过：边缘场景 batch=1，p95 比 p50 重要得多。
只报平均值的 benchmark 是不合格的。

5. 有 `--fast` 冒烟模式

第 5 课（工程实践）的核心思想：**先验证链路，再调效果。**
26 秒能走完全流程，就不用等 5 分钟才发现某个格式转换报错。

6. 一切可复现

固定随机种子、固定超参、报告里写明复现命令。

三、怎么把它写进简历

项目名称

端侧图像分类模型的全链路训练、压缩与部署

项目描述（可直接用）

> - 基于 PyTorch 从零实现 VGG 风格 CNN，在 CIFAR-10 上达到 **92.2%** 测试准确率；
>   通过数据增强与学习率预热/余弦退火解决训练不收敛问题
> - 构建 **ONNX 导出与数值对齐验证**流程（最大输出误差 **2.03e-06**），
>   建立 ONNX Runtime 基准测试框架，量化 p50/p95 延迟与峰值内存
> - 实现 **PTQ 静态量化**，模型体积 **8.41 MB -> 2.14 MB（3.92 倍）**，
>   CPU 推理延迟 **0.392 ms -> 0.125 ms（3.14 倍）**，精度仅损失 **0.35 个百分点**；
>   并识别出动态量化在卷积网络上的 **24 倍性能退化**（根因：`ConvInteger` 缺少优化内核）
> - 实现 **QAT 量化感知训练**（Fake Quant + STE），将量化损失从 -0.35pp 改善至 -0.25pp
> - 组合 **知识蒸馏 + 剪枝 + 量化**：蒸馏出 0.55M 参数的 student，
>   最终模型 **8.41 MB -> 0.565 MB（14.89 倍）**，精度仅下降 1.80pp，延迟降低 6.69 倍
> - 定量验证数据泄漏与域偏移影响：仅实验室数据训练的模型在现场测试集上仅 49.5%
>   （二分类随机水平），引入现场数据后提升至 61.2%（+11.7pp）
> - 产出**一条命令可复现**的端到端 pipeline（训练→导出→量化→基准→报告）

面试官会追问什么

上面每一条他都可能往下挖一层，所以**你必须能答出下面这些问题。**

四、11 课对应的面试问题清单

训练基础

1. 训练循环的 5 步是什么？`zero_grad()` 为什么必须存在？
2. `model.train()` 和 `model.eval()` 的区别？不切换会怎样？
3. **loss 卡在 `ln(类别数)` 说明什么？怎么排查？**（第 2 课）
4. 为什么学习率太大会导致"完全不学"？学习率预热解决什么问题？
5. 怎么判断过拟合？train/test 曲线的裂口说明什么？

导出与部署

6. 训练框架和推理框架的区别？为什么设备上不能装 PyTorch？
7. **导出 ONNX 后你怎么确认模型没坏？**（数值对齐 + 文件大小 + 真实测试集）
8. **导出后文件只有 33 KB 而模型应该 8 MB，怎么排查？**（external data）

9. 为什么导出前必须 `model.eval()`？
10. 什么是 BN 折叠？为什么推理时可以折叠？
11. ONNX 是压缩格式吗？（**不是**）

性能测量

12. **为什么 GPU 在 batch=1 时不一定比 CPU 快？**（边缘为什么用 CPU/NPU）
13. GPU 计时为什么要用 CUDA Event？
14. 为什么要 warmup？为什么看 p95 而不是只看平均？
15. 吞吐在某个 batch 后饱和说明瓶颈在哪？（内存带宽）
16. 12 核 CPU 为什么只有 2.5 倍加速？

量化

17. 为什么 INT8 能省 4 倍内存？给出换算公式。
18. 量化的仿射映射公式是什么（scale / zero_point）？
19. **校准集为什么只能用训练集？**
20. 静态量化和动态量化的区别？各自适合什么模型？
21. **为什么动态量化在 CNN 上会变慢 24 倍？**
22. 量化后精度掉了 5%，你怎么排查？
23. QAT 的 STE 是什么？为什么必需？
24. 什么时候该用 QAT，什么时候不该？

压缩

25. 软标签（dark knowledge）为什么比硬标签信息量大？温度 T 的作用？
26. **为什么非结构化剪枝通常不能加速？**
27. 结构化剪枝要处理什么依赖关系？
28. **剪枝不微调会怎样？**（92% -> 23.7%）
29. 重新设计小模型+蒸馏 vs 剪枝大模型，怎么选？
30. 同体积下，"大模型量化"和"小模型 FP32"哪个好？

数据

31. **什么是数据泄漏？怎么避免？**（按实体划分）
32. 训练集要不要覆盖部署分布？（**要**，第 9 课 B 策略 49.5%）
33. 怎么测你实验的噪声底线？差异小于噪声能下结论吗？

34. 类别不均衡时该看什么指标?
35. 数据卡片（Data Card）应该包含什么?

微调

36. 预训练权重为什么可迁移?
37. 域差异如何影响微调策略?
38. 灾难性遗忘是什么?怎么缓解?（至少 3 种手段）
39. 什么是分层学习率?

> **这 39 个问题，就是边缘 AI 岗位面试的题库。**
> 你能答上 35 个以上，就具备了通过技术面的能力。

五、结课总结

你现在会什么

能力	对应课
从零写训练循环、诊断训练问题	1–2
导出部署、数值对齐、找导出 bug	3
科学的性能基准测试	4
PTQ / QAT 量化	6–7
蒸馏、结构化剪枝	8
数据集划分方法论（泄漏/域偏移）	9
小样本迁移学习与微调	10
端到端交付与可复现	11

你还缺什么

诚实地说，这个路线**没有**覆盖：

缺口	怎么补
真实硬件部署	第 5 课一直挂着——去买一块 Jetson / 树莓派 / 安卓机，跑通 RKNN/TFLite
检测 / 分割任务	学 YOLO 系列 + NMS + mAP 评估
INT4 / 更低比特	学 GPTQ / AWQ，或者 ORT 的 INT4 支持
端侧 LLM	另一条路：llama.cpp / GGUF / MLC-LLM / ExecuTorch
模型架构设计	读 MobileNet / EfficientNet / MobileViT 论文并复现
工程化	模型版本管理、A/B 测试、线上监控、badcase 回流

建议的下一步

1. **补第 5 课**：搞一块真实设备，把 `model_int8.onnx` 真正跑起来。
   这是从"会做实验"到"能交付"的最后一步。
2. **换一个真实任务**：用你的设备采集数据，走完第 9→10→6 的全流程。
3. **把这个仓库整理好**：写清 README，让别人能一键复现。

六、最后一句

这个路线里，**真正难的不是任何一项技术，而是"不轻信结论"的习惯**：

- 第 2 课：学习率太大导致完全不学（反直觉）
- 第 4 课：GPU 在 batch=1 不占优（反直觉）
- 第 6 课：动态量化慢 24 倍（反直觉）
- 第 7 课：QAT 只提升 0.10pp（反直觉）
- 第 8 课：剪枝不微调直接崩（反直觉）
- 第 9 课：随机划分会泄漏 96%（反直觉）
- 第 10 课：两条教科书最佳实践失效（反直觉）

七次反直觉。 每一次都是靠"动手测一遍"才发现的。

> **这就是这个行业最值钱的能力：不背答案，做实验。**

恭喜你走完全部 11 课。 🎓

附录

附录：作品集与面试题库

> 一个从零构建的边缘 AI 全链路项目：**训练 → 导出 → 压缩 → 基准测试 → 端到端交付。**
> 全部代码基于 PyTorch / ONNX Runtime，**不使用任何现成的模型库。**

一、一条命令复现全部

```
cd edge-ai\lesson11_pipeline
& E:\llm-env\venv\Scripts\python.exe edge_pipeline.py --epochs 25
```

约 5 分钟，自动产出训练好的模型、ONNX 单文件、INT8 量化模型，
以及一份实测数据自动生成的报告。

实测结果（RTX 5060 Ti + 12 核 CPU）：

版本	体积	压缩比	准确率	精度变化	p50	加速
FP32	2.132 MB	1.00x	0.8930	—	0.130 ms	1.00x
INT8	**0.565 MB**	**3.77x**	**0.8940**	**+0.10 pp**	**0.070 ms**	**1.85x**

> 注意：量化后精度**没有下降反而略升**（量化起了正则化作用），
> 2000 张测试图的预测一致率 99.55%。

二、核心成果汇总

训练与部署

指标	结果	课
CIFAR-10 测试准确率（2.20M 参数 CNN）	**92.18%**	2
从零训练到 90% 走过的弯路	loss 卡死在 `ln(10)` → 定位为缺学习率预热	2

指标	结果	课
ONNX 导出数值对齐误差	**2.03e-06**	3
导出踩坑	权重被写到外部 `.data`，`.onnx` 只有 33 KB	3

性能测量

指标	结果	课
batch=1：ONNX CPU vs PyTorch GPU	**0.413 ms vs 0.462 ms（GPU 反而慢）**	4
batch=32：GPU vs CPU 吞吐	37,915 vs 2,923 张/秒（13 倍）	4

> **关键结论：边缘场景是 batch=1，所以边缘推理主力是 CPU/NPU 而不是 GPU。**

量化

指标	结果	课
PTQ 静态量化	8.412 → **2.144 MB（3.92x）**，0.392 → **0.125 ms（3.14x）**，精度 **-0.35pp**	6
动态量化的反例	**慢 24 倍**，根因：`ConvInteger` 缺优化内核 + 16 个 Mul/8 个 Add	6
QAT（Fake Quant + STE）	量化损失 **-0.35pp → -0.25pp**	7
校准集大小消融	32 张即饱和，再多无收益	6

模型压缩

手段	压缩比	精度损失	课
量化（INT8）	3.92x	**-0.35pp** ⭐	6
蒸馏（2.20M → 0.55M）	3.95x	-1.90pp	8
剪枝（-62.9% 参数）	2.68x	-2.45pp	8
蒸馏 + 量化	**14.89x**	**-1.80pp**	8

> 关键结论：**重新设计小模型 + 蒸馏，通常比剪枝大模型更划算**；
> **非结构化剪枝通常不能加速；剪枝不微调会崩（92.18% → 23.67%）。**

数据方法论

实验	结果	课
随机划分照片的数据泄漏	**96%** 的测试场景有同场景样本进入训练集	9
仅实验室数据训练的模型在现场测试集上	**49.50%**（二分类随机水平）	9
引入现场数据训练后	**61.17%（+11.67pp）**	9
实验噪声底线测定	±3pp（同配置两次运行差 3.2pp）	9
迁移学习（400 张图）	从零 49.50% → 微调 **62.67%（+13.2pp）**	10

> 关键结论：**两条教科书最佳实践（小学习率微调、少数据冻结 backbone）在本任务上失效**，定位原因为预训练域与目标域差异过大。

三、简历项目描述（可直接复制）

端侧图像分类模型的全链路训练、压缩与部署

- 基于 PyTorch **从零实现** VGG 风格 CNN，CIFAR-10 测试准确率达 **92.18%**；
  通过日志定位并修复"学习率缺预热导致 loss 卡死在 `ln(10)`、准确率恒为 10%"的训练不收敛问题
- 构建 **ONNX 导出与数值对齐验证**流程（最大输出误差 **2.03e-06**），
  识别并修复"权重被写入外部 `.data` 文件导致部署文件不完整"的问题；
  建立 ONNX Runtime 基准测试框架，量化 p50/p95 延迟与峰值内存
- 完成 **PTQ 静态量化**：模型 **8.41 MB → 2.14 MB（3.92 倍）**，
  CPU 推理 p50 **0.392 ms → 0.125 ms（3.14 倍）**，精度仅下降 **0.35 个百分点**；
  在对比实验中识别出**动态量化在卷积网络上导致 24 倍性能退化**，
  定位根因为 `ConvInteger` 缺少优化内核并引入额外 16 个 Mul / 8 个 Add 运行时反量化
- 实现 **QAT 量化感知训练**（自定义 Fake Quant + 直通估计器 STE），
  将量化精度损失从 -0.35pp 改善至 **-0.25pp**
- 组合 **知识蒸馏 + 结构化剪枝 + 量化**：以 2.20M 模型为 teacher 蒸馏出 0.55M student，
  最终模型 **8.41 MB → 0.565 MB（14.89 倍）**，精度仅下降 **1.80pp**，
  CPU 延迟降低 **6.69 倍**
- **定量验证数据泄漏与域偏移的影响**：随机划分照片导致 96% 的测试场景存在同场景泄漏；
  仅用实验室数据训练的模型在现场测试集上仅 **49.5%**（二分类随机水平），
  引入现场数据后提升至 **61.2%（+11.7pp）**；
  并测定实验噪声底线为 ±3pp，建立"按实体划分"的评测规范
- 产出**一条命令可复现**的端到端流水线（训练→导出→量化→基准→自动报告），
  5 分钟完成全流程

四、仓库结构（11 课）

课	主题	关键产出
1	训练循环的五件事	MLP on CIFAR-10，53.33%
2	CNN 到 90%	**92.18%**；`diagnose.py` 排查流程
3	导出 ONNX	对齐误差 2.03e-06
4	延迟与内存基准	4 后端 × 4 batch

课	主题	关键产出
5	部署到真实硬件	⏸ 待硬件
6	PTQ 量化	3.92x 压缩 / -0.35pp
7	QAT	-0.35pp → -0.25pp
8	剪枝与蒸馏	14.89x 压缩 / -1.80pp
9	做你自己的数据集	泄漏 96% / 域偏移 +11.67pp
10	迁移学习与微调	预训练 +13.2pp
11	端到端 pipeline	**作品集**

每一课都有：`README.md`（讲义 + 作业 + 验收标准）、可运行代码、
实测结果（`out/`）、以及图表。

五、面试题库（39 问）

训练基础

1. 训练循环的 5 步？`zero_grad()` 为什么必须存在？
2. `model.train()` / `eval()` 的区别？不切换会怎样？
3. **loss 卡在 `ln(类别数)` 说明什么？怎么排查？**
4. 为什么学习率太大会导致完全不学？预热解决什么？
5. 怎么判断过拟合？

导出与部署

6. 训练框架 vs 推理框架？为什么设备上不能装 PyTorch？
7. **导出 ONNX 后怎么确认模型没坏？**
8. **导出文件只有 33 KB 而模型应该 8 MB，怎么排查？**
9. 为什么导出前必须 `model.eval()`？
10. 什么是 BN 折叠？
11. ONNX 是压缩格式吗？

性能测量

12. **为什么 GPU 在 batch=1 时不一定比 CPU 快？**

13. GPU 计时为什么要用 CUDA Event?
14. 为什么要 warmup? 为什么看 p95?
15. 吞吐饱和说明瓶颈在哪?
16. 12 核 CPU 为什么只有 2.5 倍加速?

量化

17. 为什么 INT8 省 4 倍内存? 换算公式?
18. 量化的仿射映射公式?
19. **校准集为什么只能用训练集?**
20. 静态 vs 动态量化的区别与适用场景?
21. **为什么动态量化在 CNN 上变慢 24 倍?**
22. 量化后精度掉 5% 怎么排查?
23. STE 是什么? 为什么必需?
24. 什么时候该用 QAT?

压缩

25. 软标签为什么信息量更大? 温度 T 的作用?
26. **为什么非结构化剪枝不能加速?**
27. 结构化剪枝要处理什么依赖?
28. **剪枝不微调会怎样?**
29. 小模型+蒸馏 vs 剪枝大模型，怎么选?
30. 同体积下"大模型量化" vs "小模型 FP32"?

数据

31. **什么是数据泄漏? 怎么避免?**
32. 训练集要不要覆盖部署分布?
33. 怎么测实验噪声底线?
34. 类别不均衡看什么指标?
35. Data Card 应该包含什么?

微调

36. 预训练权重为什么可迁移?
37. 域差异如何影响微调策略?

38. 灾难性遗忘怎么缓解?

39. 什么是分层学习率?

六、能力清单与缺口

已掌握

训练循环与诊断 · ONNX 导出与对齐 · 性能基准测试 · PTQ/QAT 量化 ·
知识蒸馏 · 结构化剪枝 · 数据集划分方法论 · 小样本迁移学习 · 端到端交付

待补

缺口	怎么补
真实硬件部署	第 5 课待硬件（Jetson / 树莓派 / 安卓 / RK3588）
检测 / 分割	YOLO 系列 + NMS + mAP
INT4 及更低比特	GPTQ / AWQ
端侧 LLM	llama.cpp / GGUF / MLC-LLM / ExecuTorch
线上工程	模型版本管理、A/B 测试、badcase 回流

本作品集由 11 课的全部实测结果汇总而成。所有数字均可在各课 `out/` 目录下复现。